DSA
A to Z

Data Structures and Algorithms with Python

From absolute basics to interview-ready, with the best time and space complexity.

Complexity · Arrays · Hashing · Linked Lists · Stacks · Recursion · Sorting
Trees · Heaps · Graphs · Dynamic Programming · Greedy · Trie · Bits

Python 3 · 120 tested code listings · time and space complexity for every algorithm

Table of Contents

Start Here: How to Use This Book . . . 5

How to study each topic . . . 5

The 5-step problem-solving framework . . . 5

Input size tells you the target complexity . . . 5

Roadmap of this book . . . 6

1. Complexity Analysis . . . 7

1.1 Why complexity matters . . . 7

1.2 Big-O, Omega and Theta . . . 7

1.3 Growth rates you must know . . . 7

1.4 Rules for calculating Big-O . . . 7

1.5 Reading code: quick patterns . . . 8

1.6 Space complexity . . . 9

1.7 Amortized analysis . . . 9

1.8 Recurrences and the Master Theorem . . . 9

2. Python Toolkit for DSA . . . 11

2.1 Operation costs of built-in types . . . 11

2.2 The essential cheat sheet . . . 12

2.3 Traps that bite beginners . . . 13

2.4 Writing code that is easy to test . . . 13

3. Arrays and Strings . . . 14

3.1 Prefix sums . . . 14

3.2 Kadane's algorithm . . . 15

3.3 Two pointers . . . 15

3.4 Sliding window . . . 17

3.5 More array patterns . . . 17

3.6 String problems . . . 18

3.7 Matrix (2D array) problems . . . 19

4. Hashing: Hash Maps and Sets . . . 21

4.1 How a hash table works . . . 21

4.2 Classic hashing patterns . . . 22

5. Linked Lists . . . 24

5.1 The big four linked-list techniques . . . 25

5.2 More linked-list problems . . . 27

6. Stacks and Queues . . . 29

6.1 Stack problems . . . 30

6.2 The monotonic stack pattern . . . 30

6.3 Deque problem . . . 31

7. Recursion and Backtracking . . . 33

7.1 Recursion in three questions . . . 33

7.2 Backtracking: choose, explore, un-choose . . . 33

8. Searching and Sorting . . . 37

8.1 Linear search vs binary search . . . 37

8.2 Binary search on the answer . . . 38

8.3 Sorting algorithms . . . 39

9. Trees and Binary Search Trees . . . 42

9.1 Vocabulary . . . 42

9.2 Traversals . . . 43

9.3 Bottom-up tree recursion . . . 44

9.4 Binary Search Trees . . . 46

10. Heaps and Priority Queues . . . 48

10.1 Top-K problems . . . 49

10.2 Merging and streaming . . . 50

11. Graphs . . . 51

11.1 Vocabulary and representation . . . 51

11.2 BFS and DFS . . . 51

11.3 Cycles, ordering and bipartite graphs . . . 53

11.4 Shortest paths in weighted graphs . . . 55

11.5 Union-Find and minimum spanning trees . . . 56

12. Dynamic Programming . . . 58

12.1 What is dynamic programming? . . . 58

12.2 The DP recipe . . . 58

12.3 1D dynamic programming . . . 59

12.4 2D dynamic programming on grids . . . 61

12.5 Knapsack family . . . 61

12.6 String dynamic programming . . . 62

12.7 Bitmask DP . . . 62

12.8 DP pattern table . . . 63

13. Greedy Algorithms . . . 64

13.1 How to convince yourself greedy works . . . 64
13.2 Interval problems . . . 64
13.3 Reachability and circuits . . . 65
13.4 Value-density and coding . . . 65

14. Advanced Structures: Trie, Fenwick, Segment Tree, KMP . . . 67

14.1 Trie (prefix tree) . . . 67
14.2 Range queries with updates . . . 68
14.3 Fast string matching . . . 69

15. Bit Manipulation . . . 70

15.1 Three XOR facts that solve many puzzles . . . 70

16. Cheat Sheets, Templates and Study Plan . . . 72

16.1 Clue to technique: how to recognize the pattern . . . 72
16.2 Data structure operations at a glance . . . 73
16.3 Algorithm complexities at a glance . . . 73
16.4 Template vault . . . 73
16.5 Practice problems by topic . . . 74
16.6 A 12-week study plan . . . 75
16.7 Common mistakes checklist . . . 75
16.8 Where to go next . . . 76

START HERE

Start Here: How to Use This Book

Read this first. Five minutes now saves five weeks later.

Data Structures and Algorithms (DSA) is the skill of storing data in the right shape (data structures) and processing it with the right recipe (algorithms) so that programs are **fast** (time complexity) and **lean** (space complexity). Every coding interview, competitive contest and high-performance system rests on it.

This book teaches DSA from absolute basics to advanced topics. Every topic follows the same rhythm: **intuition in plain words, then clean Python code, then the time and space complexity**. All code in this book was executed against test cases before being included.

How to study each topic

1. **Read the idea** and try to explain it to an imaginary friend in two sentences.
2. **Close the book and code it** from scratch. Struggling here is where learning happens.
3. **Compare** with the listing. Note what you missed (edge case? off-by-one?).
4. **State the complexity** out loud: time and space, and why.
5. **Practice 3 to 5 variants** (see the practice list in the last chapter) before moving on.

The 5-step problem-solving framework

Step	What to do
1. Understand	Restate the problem. Identify inputs, outputs and edge cases: empty input, one element, duplicates, negative numbers, very large n.
2. Brute force	State the simplest correct solution and its complexity. Never skip this; it is your safety net and your baseline.
3. Optimize	Find the bottleneck (usually repeated work or a nested loop). Apply a pattern: hash map, two pointers, sorting, heap, binary search, DP, graph traversal.
4. Code	Write clean code with meaningful names. Use helper functions. Do not optimize prematurely inside the code.
5. Verify	Dry-run on a small example and one edge case. Finally state the time and space complexity.

Input size tells you the target complexity

Look at the constraints before choosing an algorithm. The table below is your compass.

Input size n	Target complexity	Typical techniques
n <= 10	O(n!)	Permutations, brute-force backtracking
n <= 20	O(2^n)	Subsets, bitmask DP, meet in the middle
n <= 100	O(n^3)	Floyd-Warshall, triple-loop DP
n <= 5,000	O(n^2)	2D DP, checking all pairs
n <= 10^5 to 10^6	O(n log n) or O(n)	Sorting, heaps, binary search, hashing, two pointers, sliding window
n <= 10^9 or more	O(log n) or O(1)	Binary search on answer, math formulas, bit tricks

Python reality check. C++ runs about 10^8 simple operations per second; Python manages roughly 10^7. In Python aim one notch lower than the table when n is large, and lean on built-ins (sorted, heapq, Counter) which are implemented in C.

Roadmap of this book

Part	Topics	After this you can solve
Foundations	Complexity analysis, Python toolkit	Judge any solution; write fast Python
Linear structures	Arrays, strings, hashing, linked lists, stacks, queues	Most easy and medium array/string problems
Core techniques	Recursion, backtracking, searching, sorting	Subsets, permutations, N-Queens, binary-search-on-answer
Non-linear structures	Trees, heaps, graphs	Traversals, shortest paths, scheduling, top-K problems
Optimization	Dynamic programming, greedy	Knapsack, LCS, edit distance, intervals, Huffman
Advanced	Trie, Fenwick, segment tree, KMP, bit manipulation	Prefix search, range queries, fast string matching
Mastery	Cheat sheets, templates, 12-week plan	A repeatable path to interview-level fluency

CHAPTER 1

Complexity Analysis

How to measure and compare algorithms without a stopwatch.

1.1 Why complexity matters

Imagine searching for a name among 1,000,000 records. Checking one by one takes up to a million steps. Halving the search range each time (binary search) takes about 20 steps. Same problem, same computer, a 50,000x difference. Complexity analysis lets you predict that difference *before* writing code.

1.2 Big-O, Omega and Theta

- **Big-O, O(f(n))**: an *upper bound*. 'The algorithm never grows faster than f(n)'. Most commonly quoted for the worst case.
- **Big-Omega, Omega(f(n))**: a *lower bound*. 'It takes at least f(n)'. Often used for the best case.
- **Big-Theta, Theta(f(n))**: a *tight bound*. Upper and lower bounds match.
- **Best / average / worst case** are different inputs, not different notations. Linear search: best O(1) (first item), worst O(n) (last item or missing).

1.3 Growth rates you must know

Class	Name	n = 10	n = 1,000	n = 1,000,000	Typical example
O(1)	Constant	1	1	1	Array index, dict lookup
O(log n)	Logarithmic	3	10	20	Binary search, heap push
O(n)	Linear	10	1,000	10^6	Scan an array
O(n log n)	Linearithmic	33	10^4	2 x 10^7	Merge sort, heap sort
O(n^2)	Quadratic	100	10^6	10^12 (too slow)	Bubble sort, all pairs
O(2^n)	Exponential	1,024	astronomical	impossible	All subsets
O(n!)	Factorial	3,628,800	impossible	impossible	All permutations

1.4 Rules for calculating Big-O

- **Drop constants**: O(2n) and O(n/2) are both O(n).
- **Drop lower-order terms**: O(n^2 + n + 7) is O(n^2). The biggest term dominates for large n.
- **Sequential steps add**: a loop of n followed by a loop of m is O(n + m).
- **Nested steps multiply**: a loop of n containing a loop of m is O(n * m).
- **Different inputs, different letters**: two arrays of sizes a and b give O(a * b), not O(n^2).
- **Recursion**: count (number of calls) x (work per call).

1.5 Reading code: quick patterns

Code shape	Complexity
for i in range(n)	O(n)
for i in range(n): for j in range(n)	O(n^2)
for i in range(n): for j in range(i)	O(n^2) (sum 1+2+...+n = n(n+1)/2)
while i < n: i *= 2 (or n //= 2)	O(log n)
for i in range(n): (inner loop that does i *= 2)	O(n log n)
Recursion with 2 calls on n-1 (naive Fibonacci)	O(2^n)
Halve the problem, O(n) work to combine (merge sort)	O(n log n)
Sorting inside the function (sorted(), list.sort())	O(n log n)

Five classic shapes in code

Study each function and say its complexity before reading the comment. Notice that `has_duplicate_fast` buys speed with memory: that is the time-space trade-off you will meet again and again.

```python
def get_first(arr):                 # O(1) time, O(1) space
    return arr[0]


def total(arr):                     # O(n) time, O(1) space
    s = 0
    for x in arr:
        s += x
    return s


def count_halvings(n):              # O(log n) time, O(1) space
    steps = 0
    while n > 1:
        n //= 2
        steps += 1
    return steps


def has_duplicate_slow(arr):        # O(n^2) time, O(1) space
    for i in range(len(arr)):
        for j in range(i + 1, len(arr)):
            if arr[i] == arr[j]:
                return True
    return False


def has_duplicate_fast(arr):        # O(n) time, O(n) space (trade memory for speed)
    seen = set()
    for x in arr:
        if x in seen:
            return True
        seen.add(x)
    return False
```

TIME O(1) to O(n^2)

SPACE O(1) or O(n)

Fibonacci: three complexities for one problem

Naive recursion recomputes the same subproblems over and over (exponential). Caching results (**memoization**) makes each value computed once. Keeping only the last two values removes the cache entirely.

```python
from functools import lru_cache


def fib_slow(n):                        # O(2^n) time, O(n) stack space
    return n if n < 2 else fib_slow(n - 1) + fib_slow(n - 2)


@lru_cache(maxsize=None)
def fib_memo(n):                        # O(n) time, O(n) space
    return n if n < 2 else fib_memo(n - 1) + fib_memo(n - 2)


def fib_best(n):                        # O(n) time, O(1) space
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

TIME O(2^n) / O(n) / O(n) **SPACE** O(n) / O(n) / O(1)

1.6 Space complexity

Space complexity counts the **extra memory** an algorithm needs, beyond the input itself (called *auxiliary space*). Three things to count:

- **New data structures** you create: a set of n items is O(n); a 2D table of n x m is O(n * m).
- **Recursion stack**: each active call uses memory. Recursion depth d costs O(d) space, even if you create no variables. Naive DFS on a path-shaped tree of n nodes uses O(n) stack.
- **Copies**: slicing `arr[1:]` or concatenating strings creates a new object of size O(k).

1.7 Amortized analysis

Some operations are cheap most of the time and expensive occasionally. Python's `list.append` is O(1) *amortized*: when the internal array is full, it allocates roughly double the space and copies everything (O(n)), but that happens so rarely that the average cost per append over many appends is still O(1). The same idea applies to hash-table resizing and to the two-stack queue in the Stacks chapter.

1.8 Recurrences and the Master Theorem

A divide-and-conquer algorithm is described by **T(n) = a * T(n/b) + O(n^d)**: it makes *a* recursive calls on pieces of size n/b and does O(n^d) extra work. Compare a with b^d:

Case	Result	Meaning
a < b^d	O(n^d)	Top-level work dominates
a = b^d	O(n^d log n)	Every level costs the same; there are log n levels
a > b^d	O(n^(log_b a))	Leaves dominate

Recurrence	Complexity	Algorithm
T(n) = T(n/2) + O(1)	O(log n)	Binary search
T(n) = 2T(n/2) + O(1)	O(n)	Tree traversal, max of array by halves
T(n) = 2T(n/2) + O(n)	O(n log n)	Merge sort, quicksort (average)
T(n) = T(n-1) + O(1)	O(n)	Simple recursion (factorial)
T(n) = T(n-1) + O(n)	O(n^2)	Quicksort worst case, selection sort
T(n) = 2T(n-1) + O(1)	O(2^n)	Naive Fibonacci, subsets

> **Golden rule.** When you finish any solution, say: 'Time is O(...) because ..., space is O(...) because ...'. Interviewers listen for the *because*.

CHAPTER 2

Python Toolkit for DSA

The built-ins that save you hundreds of lines, and the traps that cost you hours.

2.1 Operation costs of built-in types

Knowing what each built-in operation costs is half of writing fast code. Most beginner slowness comes from hidden O(n) operations inside loops.

Type	Operation	Cost	Note
list	a[i], a[i] = x, len(a)	O(1)	Dynamic array
list	append(x), pop()	O(1) amortized	Operate at the end
list	insert(i, x), pop(0), del a[i]	O(n)	Shifts elements; avoid pop(0), use deque
list	x in a, a.index(x), a.count(x)	O(n)	Linear scan
list	a[i:j] (slice)	O(j - i)	Creates a copy
list	sort(), sorted(a)	O(n log n)	Timsort, stable
dict / set	get, set, delete, in	O(1) average	Worst case O(n) with many collisions
deque	append, appendleft, pop, popleft	O(1)	Use as queue; a[i] in the middle is O(n)
heapq	heappush, heappop	O(log n)	Min-heap only
heapq	heapify(list)	O(n)	Builds a heap in place
str	s[i], len(s)	O(1)	Immutable
str	s + t, s += c	O(len(s) + len(t))	New string each time; in loops use ''.join(parts)
bisect	bisect_left, bisect_right	O(log n)	List must be sorted; insort is O(n)

2.2 The essential cheat sheet

```python
from collections import deque, Counter, defaultdict
import heapq
import bisect
import math

# list as a stack
stack = []
stack.append(1)
stack.append(2)
top = stack.pop()                       # 2, O(1)

# deque as a queue
q = deque([1, 2, 3])
q.append(4)
first = q.popleft()                     # 1, O(1)

# Counter and defaultdict
freq = Counter("banana")                # a:3, n:2, b:1
graph = defaultdict(list)
graph[1].append(2)                      # no KeyError for missing keys

# min-heap, and the negative trick for a max-heap
h = [5, 1, 3]
heapq.heapify(h)
smallest = heapq.heappop(h)             # 1
max_heap = []
heapq.heappush(max_heap, -10)
heapq.heappush(max_heap, -4)
largest = -heapq.heappop(max_heap)      # 10

# binary search in a sorted list
arr = [1, 3, 5, 7]
i = bisect.bisect_left(arr, 5)          # 2 (first index with arr[i] >= 5)

# infinity, custom sorting
INF = math.inf
words = ["bb", "a", "ccc"]
words.sort(key=len)                     # ['a', 'bb', 'ccc']
pairs = sorted([(2, "b"), (1, "z")], key=lambda p: p[0])
```

2.3 Traps that bite beginners

```python
# Trap 1: building a 2D grid
grid_bad = [[0] * 3] * 3               # WRONG: 3 references to ONE row
grid_good = [[0] * 3 for _ in range(3)] # RIGHT: 3 independent rows
grid_bad[0][0] = 1
grid_good[0][0] = 1

# Trap 2: integer division and modulo with negatives
a = -7 // 2                            # -4 (floors toward minus infinity)
b = int(-7 / 2)                        # -3 (truncates toward zero)
c = -7 % 3                             # 2 (never negative for a positive divisor)

# Trap 3: mutable default argument (shared between calls!)
def bad(x, acc=[]):
    acc.append(x)
    return acc

def good(x, acc=None):
    if acc is None:
        acc = []
    acc.append(x)
    return acc

# Trap 4: building strings in a loop is O(n^2); join is O(n)
parts = [str(i) for i in range(5)]
s = "".join(parts)
```

Recursion limit. Python stops recursion at about 1000 frames. For deep DFS or recursion over 10^5 items, either convert to an iterative version with an explicit stack (recommended) or raise the limit with `sys.setrecursionlimit(10**6)` and be aware that very deep recursion can still crash the interpreter.

2.4 Writing code that is easy to test

- Write functions that **take input and return output** instead of reading and printing; they can be tested with `assert`.
- Test the **edge cases** first: empty, single element, all equal, already sorted, negatives.
- Compare a clever solution to a slow-but-obviously-correct brute force on random small inputs.

CHAPTER 3

Arrays and Strings

The most common interview topic. Master these patterns and half the battle is won.

An **array** stores items in consecutive memory, so reading `a[i]` is O(1). A Python `list` is a dynamic array. Inserting or deleting in the middle is O(n) because items must shift. A Python `str` is an immutable array of characters. Almost every array problem is solved by one of the patterns in this chapter: **prefix sums, Kadane, two pointers, sliding window, Dutch flag**.

Reverse an array in place

Swap the two ends and walk inward. Using indices instead of `arr[::-1]` avoids creating a copy, so extra space is O(1).

```
def reverse_in_place(arr):
    l, r = 0, len(arr) - 1
    while l < r:
        arr[l], arr[r] = arr[r], arr[l]
        l += 1
        r -= 1
    return arr
```

TIME O(n) **SPACE** O(1)

Rotate an array right by k (three reversals)

Reverse the whole array, then reverse the first k items, then the rest. Example for k=3: 1234567 becomes 7654321, then 567 4321, then 5671234. No extra array needed.

```
def rotate_right(arr, k):
    if not arr:
        return arr
    n = len(arr)
    k %= n

    def rev(l, r):
        while l < r:
            arr[l], arr[r] = arr[r], arr[l]
            l += 1
            r -= 1

    rev(0, n - 1)
    rev(0, k - 1)
    rev(k, n - 1)
    return arr
```

TIME O(n) **SPACE** O(1)

3.1 Prefix sums

Idea: precompute `pre[i]` = sum of the first i items. Then any range sum is `pre[r+1] - pre[l]` in O(1). Use it whenever you answer many range-sum questions on a static array. The same trick works for counts, XOR and even 2D matrices.

Range sum query

Building costs O(n) once; every query afterwards is O(1) instead of O(n).

```python
class PrefixSum:
    def __init__(self, nums):
        self.pre = [0]
        for x in nums:
            self.pre.append(self.pre[-1] + x)

    def range_sum(self, l, r):           # inclusive l..r
        return self.pre[r + 1] - self.pre[l]
```

TIME O(n) build, O(1) query **SPACE** O(n)

3.2 Kadane's algorithm

Maximum subarray sum

At each element decide: extend the running subarray, or start fresh here. `cur = max(x, cur + x)`. If the running sum has become a burden (negative), dropping it is always better. Track the best value seen.

```python
def max_subarray(nums):
    best = cur = nums[0]
    for x in nums[1:]:
        cur = max(x, cur + x)
        best = max(best, cur)
    return best
```

TIME O(n) **SPACE** O(1)

3.3 Two pointers

Idea: keep two indices and move them based on a rule, so that each index only travels once. It turns many O(n^2) pair-search problems into O(n). Two flavors: **opposite ends** moving inward (sorted arrays, palindromes) and **same direction** where a fast pointer reads and a slow pointer writes (in-place filtering).

Two sum in a sorted array

If the current pair sums too small, only moving the left pointer right can increase the sum; if too big, move the right pointer left. Each step discards one candidate for good.

```python
def two_sum_sorted(nums, target):
    l, r = 0, len(nums) - 1
    while l < r:
        s = nums[l] + nums[r]
        if s == target:
            return [l, r]
        if s < target:
            l += 1
        else:
            r -= 1
    return []
```

TIME O(n) **SPACE** O(1)

Remove duplicates from a sorted array (in place)

The slow pointer `w` marks where the next unique value goes; the fast pointer scans. Returns the new length.

```python
def remove_duplicates(nums):
    if not nums:
        return 0
    w = 1
    for i in range(1, len(nums)):
        if nums[i] != nums[w - 1]:
```

```
            nums[w] = nums[i]
            w += 1
    return w
```

TIME O(n) **SPACE** O(1)

Move all zeroes to the end

Same read/write pattern: swap every non-zero element forward to position `w`. Order of non-zero items is preserved.

```
def move_zeroes(nums):
    w = 0
    for i in range(len(nums)):
        if nums[i] != 0:
            nums[w], nums[i] = nums[i], nums[w]
            w += 1
    return nums
```

TIME O(n) **SPACE** O(1)

Container with most water

Area is limited by the shorter wall. Start with the widest container, then move the *shorter* wall inward: moving the taller one can never help because width shrinks while height is still capped by the short wall.

```
def max_area(h):
    l, r, best = 0, len(h) - 1, 0
    while l < r:
        best = max(best, min(h[l], h[r]) * (r - l))
        if h[l] < h[r]:
            l += 1
        else:
            r -= 1
    return best
```

TIME O(n) **SPACE** O(1)

3Sum (all unique triplets that add to zero)

Sort, fix one number, then solve two-sum with two pointers on the rest. Skip duplicate values so each triplet appears once.

```
def three_sum(nums):
    nums.sort()
    res = []
    for i in range(len(nums) - 2):
        if i > 0 and nums[i] == nums[i - 1]:
            continue                    # skip duplicate first numbers
        l, r = i + 1, len(nums) - 1
        while l < r:
            s = nums[i] + nums[l] + nums[r]
            if s < 0:
                l += 1
            elif s > 0:
                r -= 1
            else:
                res.append([nums[i], nums[l], nums[r]])
                l += 1
                while l < r and nums[l] == nums[l - 1]:
                    l += 1              # skip duplicate second numbers
                r -= 1
    return res
```

TIME O(n^2) **SPACE** O(1) extra (sorting aside)

3.4 Sliding window

Idea: a window `[left, right]` slides across the array. Instead of recomputing the window's answer from scratch (O(k)), update it by adding the new item and removing the old one (O(1)). Use a **fixed-size** window when k is given; use a **variable-size** window (grow right, shrink left while a condition is broken) for 'longest/shortest subarray with property X'.

Maximum sum of a subarray of size k (fixed window)

Compute the first window once. Then slide: add the entering element, subtract the leaving one.

```python
def max_sum_k(nums, k):
    window = sum(nums[:k])
    best = window
    for i in range(k, len(nums)):
        window += nums[i] - nums[i - k]
        best = max(best, window)
    return best
```

TIME O(n) **SPACE** O(1)

Longest substring without repeating characters (variable window)

Remember the last index of every character. When the new character already appears inside the window, jump `left` just past its previous position.

```python
def longest_unique(s):
    last = {}
    left = best = 0
    for right, ch in enumerate(s):
        if ch in last and last[ch] >= left:
            left = last[ch] + 1
        last[ch] = right
        best = max(best, right - left + 1)
    return best
```

TIME O(n) **SPACE** O(min(n, alphabet))

Smallest subarray with sum at least target (variable window)

Grow the window by moving `right`; whenever the sum reaches the target, record the length and shrink from the left as far as possible. Works because all numbers are positive.

```python
def min_subarray_len(target, nums):
    left = total = 0
    best = float("inf")
    for right, x in enumerate(nums):
        total += x
        while total >= target:
            best = min(best, right - left + 1)
            total -= nums[left]
            left += 1
    return 0 if best == float("inf") else best
```

TIME O(n) **SPACE** O(1)

3.5 More array patterns

Sort colors: Dutch national flag

Three regions: `[0, lo)` are 0s, `[lo, mid)` are 1s, `(hi, end]` are 2s; `[mid, hi]` is unexplored. One pass, no counting array.

```python
def sort_colors(nums):
    lo, mid, hi = 0, 0, len(nums) - 1
    while mid <= hi:
        if nums[mid] == 0:
            nums[lo], nums[mid] = nums[mid], nums[lo]
            lo += 1
            mid += 1
        elif nums[mid] == 1:
            mid += 1
        else:
            nums[mid], nums[hi] = nums[hi], nums[mid]
            hi -= 1
    return nums
```

TIME O(n) **SPACE** O(1)

Product of array except self (no division)

Answer at i = (product of everything left of i) x (product of everything right of i). Compute left products in one pass and multiply in right products in a second pass.

```python
def product_except_self(nums):
    n = len(nums)
    res = [1] * n
    left = 1
    for i in range(n):
        res[i] = left
        left *= nums[i]
    right = 1
    for i in range(n - 1, -1, -1):
        res[i] *= right
        right *= nums[i]
    return res
```

TIME O(n) **SPACE** O(1) extra (output excluded)

3.6 String problems

Valid palindrome (ignore punctuation and case)

Two pointers from both ends, skipping non-alphanumeric characters.

```python
def is_palindrome(s):
    l, r = 0, len(s) - 1
    while l < r:
        while l < r and not s[l].isalnum():
            l += 1
        while l < r and not s[r].isalnum():
            r -= 1
        if s[l].lower() != s[r].lower():
            return False
        l += 1
        r -= 1
    return True
```

TIME O(n) **SPACE** O(1)

Valid anagram

Two strings are anagrams exactly when their character counts match.

```python
from collections import Counter


def is_anagram(s, t):
    return Counter(s) == Counter(t)
```

TIME O(n) **SPACE** O(alphabet)

Longest palindromic substring (expand around center)

Every palindrome has a center: a character (odd length) or a gap between two characters (even length). Try all 2n-1 centers and expand outward while both sides match.

```
def longest_palindrome(s):
    start, best = 0, 0

    def expand(l, r):
        while l >= 0 and r < len(s) and s[l] == s[r]:
            l -= 1
            r += 1
        return l + 1, r - l - 1          # (start, length)

    for i in range(len(s)):
        for a, b in ((i, i), (i, i + 1)):
            st, ln = expand(a, b)
            if ln > best:
                start, best = st, ln
    return s[start:start + best]
```

TIME O(n^2) **SPACE** O(1)

Reverse the words in a sentence

`split()` without arguments collapses any run of spaces, so extra spaces vanish automatically.

```
def reverse_words(s):
    return " ".join(reversed(s.split()))
```

TIME O(n) **SPACE** O(n)

3.7 Matrix (2D array) problems

Rotate an image 90 degrees clockwise (in place)

Rotating clockwise equals **transpose** (swap rows and columns) followed by **reversing each row**.

```
def rotate_matrix(m):
    n = len(m)
    for i in range(n):
        for j in range(i + 1, n):
            m[i][j], m[j][i] = m[j][i], m[i][j]
    for row in m:
        row.reverse()
    return m
```

TIME O(n^2) **SPACE** O(1)

Spiral order traversal

Maintain four boundaries (top, bottom, left, right). Walk the top row, right column, bottom row and left column, shrinking the boundary after each side. Guard the last two sides against single-row or single-column leftovers.

```
def spiral_order(m):
    if not m:
        return []
    res = []
    top, bottom, left, right = 0, len(m) - 1, 0, len(m[0]) - 1
    while top <= bottom and left <= right:
        for j in range(left, right + 1):
            res.append(m[top][j])
        top += 1
```

```
        for i in range(top, bottom + 1):
            res.append(m[i][right])
        right -= 1
        if top <= bottom:
            for j in range(right, left - 1, -1):
                res.append(m[bottom][j])
            bottom -= 1
        if left <= right:
            for i in range(bottom, top - 1, -1):
                res.append(m[i][left])
            left += 1
    return res
```

TIME O(rows * cols)

SPACE O(1) extra

CHAPTER 4

Hashing: Hash Maps and Sets

The single most useful trick for turning O(n^2) into O(n).

4.1 How a hash table works

A hash table stores key-value pairs in an array of **buckets**. A **hash function** turns the key into a number; that number modulo the array size picks the bucket. Lookup, insert and delete are therefore O(1) on average, no matter how many items are stored.

- **Collision**: two keys land in the same bucket. **Chaining** keeps a small list per bucket; **open addressing** probes for the next free slot (Python dicts use open addressing).
- **Load factor** = items / buckets. When it grows past about 0.7, the table doubles its size and re-inserts everything (amortized O(1) per insert).
- Keys must be **hashable** (immutable): numbers, strings, tuples are fine; lists and dicts are not. Use `tuple(...)` to use a list as a key.
- Worst case is O(n) if every key collides, which is why good hash functions spread keys evenly.

```
 key     hash(key) % 4      buckets (chaining)
-----    --------------     ------------------------------
"cat"         1             [0] -> empty
"dog"         3             [1] -> ("cat", 5)
"owl"         1             [2] -> empty
                            [3] -> ("dog", 2) -> ("owl", 9)   <- collisions chain
```

Build your own hash map (chaining + resizing)

This is a teaching version of what `dict` does internally. Notice the resize step that keeps buckets short.

```python
class MyHashMap:
    def __init__(self, capacity=8):
        self.cap = capacity
        self.size = 0
        self.buckets = [[] for _ in range(capacity)]

    def _idx(self, key):
        return hash(key) % self.cap

    def put(self, key, value):
        bucket = self.buckets[self._idx(key)]
        for pair in bucket:
            if pair[0] == key:
                pair[1] = value              # update existing key
                return
        bucket.append([key, value])
        self.size += 1
        if self.size / self.cap > 0.75:      # load factor too high
            self._resize()

    def get(self, key, default=None):
        for k, v in self.buckets[self._idx(key)]:
            if k == key:
                return v
        return default

    def remove(self, key):
        bucket = self.buckets[self._idx(key)]
        for i, pair in enumerate(bucket):
            if pair[0] == key:
```

```
                bucket.pop(i)
                self.size -= 1
                return True
        return False

    def _resize(self):
        old = self.buckets
        self.cap *= 2
        self.buckets = [[] for _ in range(self.cap)]
        for bucket in old:
            for k, v in bucket:
                self.buckets[self._idx(k)].append([k, v])
```

TIME O(1) average per operation **SPACE** O(n)

4.2 Classic hashing patterns

Two sum (the pattern that started it all)

For each number x, the partner is `target - x`. Keep a dictionary of numbers already seen with their indices, and ask 'have I seen the partner?' in O(1) instead of scanning.

```
def two_sum(nums, target):
    seen = {}                                # value -> index
    for i, x in enumerate(nums):
        if target - x in seen:
            return [seen[target - x], i]
        seen[x] = i
    return []
```

TIME O(n) **SPACE** O(n)

Group anagrams

Anagrams become identical when their letters are sorted. Use the sorted letters as the dictionary key and collect words under it.

```
from collections import defaultdict


def group_anagrams(words):
    groups = defaultdict(list)
    for w in words:
        groups[tuple(sorted(w))].append(w)
    return list(groups.values())
```

TIME O(n * k log k) for n words of length k **SPACE** O(n * k)

Subarray sum equals K (prefix sum + hash map)

If the running sum up to j is `S` and the running sum up to some earlier i was `S - k`, then the subarray between them sums to k. Count how many earlier prefix sums equal `S - k`. Seed with `{0: 1}` so subarrays starting at index 0 are counted.

```
def subarray_sum(nums, k):
    count = running = 0
    seen = {0: 1}                            # prefix sum -> how many times seen
    for x in nums:
        running += x
        count += seen.get(running - k, 0)
        seen[running] = seen.get(running, 0) + 1
    return count
```

TIME O(n) **SPACE** O(n)

Longest consecutive sequence

Put all numbers in a set. Only start counting from a number `x` that has no `x - 1` (the true start of a run), then extend upward. Each number is visited at most twice, so the total is O(n) even without sorting.

```python
def longest_consecutive(nums):
    s = set(nums)
    best = 0
    for x in s:
        if x - 1 not in s:                  # x begins a sequence
            y = x
            while y + 1 in s:
                y += 1
            best = max(best, y - x + 1)
    return best
```

TIME O(n) **SPACE** O(n)

First non-repeating character

Count frequencies in one pass, then find the first character whose count is 1 in a second pass.

```python
from collections import Counter


def first_unique_char(s):
    freq = Counter(s)
    for i, ch in enumerate(s):
        if freq[ch] == 1:
            return i
    return -1
```

TIME O(n) **SPACE** O(alphabet)

LRU cache (hash map + ordering)

A Least-Recently-Used cache needs O(1) get, O(1) put and O(1) eviction of the oldest item. `OrderedDict` is a hash map that also remembers order, so `move_to_end` marks an item as freshly used and `popitem(last=False)` evicts the stalest. (The classic hand-built version is a hash map plus a doubly linked list; this does the same job.)

```python
from collections import OrderedDict


class LRUCache:
    def __init__(self, capacity):
        self.cap = capacity
        self.data = OrderedDict()

    def get(self, key):
        if key not in self.data:
            return -1
        self.data.move_to_end(key)          # mark as most recently used
        return self.data[key]

    def put(self, key, value):
        if key in self.data:
            self.data.move_to_end(key)
        self.data[key] = value
        if len(self.data) > self.cap:
            self.data.popitem(last=False)   # evict least recently used
```

TIME O(1) per operation **SPACE** O(capacity)

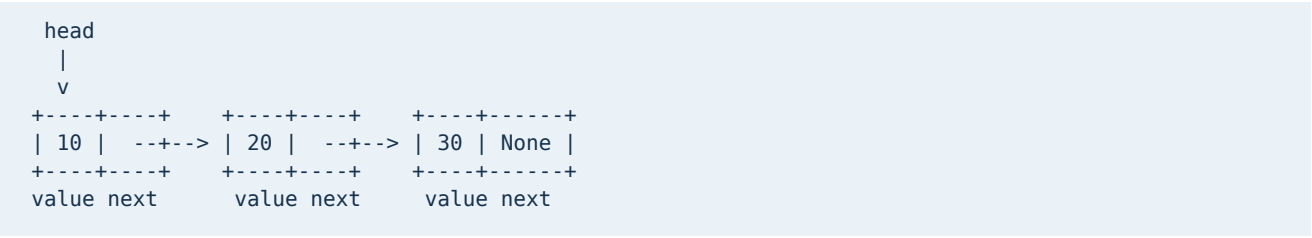
When you see: 'find a pair/duplicate/count', 'have I seen this before', 'group by property', or 'frequency'. The answer is usually a dictionary or set. Cost: O(n) extra space to save a whole loop of time.

CHAPTER 5

Linked Lists

Nodes connected by pointers: cheap insertion, expensive random access.

A **linked list** is a chain of nodes; each node stores a value and a pointer to the next node. Unlike an array there is no index: to reach the 5th item you walk 4 links (O(n)). In exchange, inserting or deleting after a known node is O(1) because only pointers change; nothing shifts.



Operation	Array (list)	Singly linked list
Access by index	O(1)	O(n)
Search by value	O(n)	O(n)
Insert / delete at front	O(n)	O(1)
Insert / delete at end	O(1) amortized / O(1)	O(n), or O(1) with a tail pointer
Insert / delete after a known node	O(n)	O(1)
Memory per item	Value only	Value + pointer

Node class and test helpers

Every linked-list problem starts with this node. The two helpers convert between Python lists and linked lists so that examples are easy to write and check. Later listings in this chapter reuse them.

```python
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next


def build(values):                          # list -> linked list
    dummy = ListNode()
    cur = dummy
    for v in values:
        cur.next = ListNode(v)
        cur = cur.next
    return dummy.next


def to_list(head):                          # linked list -> list
    out = []
    while head:
        out.append(head.val)
        head = head.next
    return out
```

TIME O(n) **SPACE** O(n)

Dummy-node trick: create a fake node before the head so that inserting or deleting the real head needs no special case. You will see `dummy = ListNode(0, head)` throughout this chapter.

A complete singly linked list

Push at the front is O(1). Pushing at the back walks to the end, O(n); keep a tail pointer to make it O(1). Delete uses a `prev` pointer so we can re-link around the removed node.

```python
class LinkedList:
    def __init__(self):
        self.head = None
        self.size = 0

    def push_front(self, val):                # O(1)
        self.head = ListNode(val, self.head)
        self.size += 1

    def push_back(self, val):                 # O(n)
        node = ListNode(val)
        if not self.head:
            self.head = node
        else:
            cur = self.head
            while cur.next:
                cur = cur.next
            cur.next = node
        self.size += 1

    def delete(self, val):                    # O(n): removes first match
        dummy = ListNode(0, self.head)
        prev, cur = dummy, self.head
        while cur:
            if cur.val == val:
                prev.next = cur.next
                self.size -= 1
                break
            prev, cur = cur, cur.next
        self.head = dummy.next

    def find(self, val):                      # O(n)
        cur = self.head
        while cur:
            if cur.val == val:
                return True
            cur = cur.next
        return False
```

TIME See comments **SPACE** O(1) per operation

5.1 The big four linked-list techniques

Reverse a linked list

Walk once and flip each `next` pointer backwards. Save `cur.next` first or you lose the rest of the list. The recursive version is shorter to write but uses O(n) stack.

```python
def reverse_list(head):
    prev = None
    cur = head
    while cur:
        nxt = cur.next                        # 1. remember the rest
        cur.next = prev                       # 2. flip the pointer
        prev = cur                            # 3. advance prev
        cur = nxt                             # 4. advance cur
    return prev


def reverse_list_recursive(head):             # O(n) time, O(n) stack
    if not head or not head.next:
        return head
```

```
    new_head = reverse_list_recursive(head.next)
    head.next.next = head
    head.next = None
    return new_head
```

TIME O(n) **SPACE** O(1) iterative, O(n) recursive

Find the middle node (slow and fast pointers)

Fast moves two steps for every one step of slow. When fast reaches the end, slow is at the middle. This **tortoise-and-hare** idea powers the next two problems as well.

```
def middle_node(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow
```

TIME O(n) **SPACE** O(1)

Detect a cycle and find where it starts (Floyd's algorithm)

If a cycle exists, fast eventually laps slow and they meet. To find the cycle's entrance, restart one pointer from the head and move both one step at a time; they meet exactly at the entrance (a neat consequence of the distances involved).

```
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow is fast:
            return True
    return False


def cycle_start(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow is fast:
            slow = head
            while slow is not fast:
                slow = slow.next
                fast = fast.next
            return slow
    return None
```

TIME O(n) **SPACE** O(1)

Merge two sorted lists

Compare the two front nodes, attach the smaller to the output tail, advance. A dummy head removes the empty-output special case. No new nodes are created: we re-link existing ones.

```
def merge_two_lists(a, b):
    dummy = cur = ListNode()
    while a and b:
        if a.val <= b.val:
            cur.next, a = a, a.next
        else:
            cur.next, b = b, b.next
```

```
        cur = cur.next
    cur.next = a or b                       # attach whichever list remains
    return dummy.next
```

TIME O(n + m) **SPACE** O(1)

5.2 More linked-list problems

Remove the n-th node from the end (one pass)

Move `fast` n steps ahead, then move both until fast reaches the last node. Now `slow` sits just before the target. The dummy node handles removing the head.

```
def remove_nth_from_end(head, n):
    dummy = ListNode(0, head)
    fast = slow = dummy
    for _ in range(n):
        fast = fast.next
    while fast.next:
        fast = fast.next
        slow = slow.next
    slow.next = slow.next.next
    return dummy.next
```

TIME O(n) **SPACE** O(1)

Add two numbers stored as reversed digit lists

Exactly how you add on paper: digit by digit with a carry. Digits are stored least-significant first, so we can add from the head.

```
def add_two_numbers(a, b):
    dummy = cur = ListNode()
    carry = 0
    while a or b or carry:
        total = carry + (a.val if a else 0) + (b.val if b else 0)
        carry, digit = divmod(total, 10)
        cur.next = ListNode(digit)
        cur = cur.next
        a = a.next if a else None
        b = b.next if b else None
    return dummy.next
```

TIME O(max(n, m)) **SPACE** O(max(n, m))

Palindrome linked list

Find the middle, reverse the second half in place, and compare both halves. Uses three tricks from this chapter at once.

```
def is_palindrome_list(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    prev = None                             # reverse the second half
    while slow:
        nxt = slow.next
        slow.next = prev
        prev = slow
        slow = nxt
    left, right = head, prev
    while right:
        if left.val != right.val:
```

```
            return False
        left, right = left.next, right.next
    return True
```

TIME O(n) **SPACE** O(1)

Doubly linked lists add a `prev` pointer to each node, allowing O(1) deletion of a node you already hold and backward traversal. They are the engine inside LRU caches and browser history. **Circular lists** make the tail point back to the head (round-robin scheduling).

CHAPTER 6

Stacks and Queues

Two disciplines for ordering work: last-in-first-out and first-in-first-out.

A **stack** is LIFO: the last item pushed is the first popped, like a pile of plates. Use a Python list: `append` to push, `pop` to pop, `[-1]` to peek, all O(1). A **queue** is FIFO: first in, first out, like a line at a ticket counter. Use `collections.deque` (`append` and `popleft` are O(1)); never use `list.pop(0)`, which is O(n).

Structure	Order	Python tool	Classic uses
Stack	Last in, first out	list	Undo, matching brackets, DFS, expression evaluation, monotonic stack
Queue	First in, first out	deque	BFS, task scheduling, buffering
Deque	Both ends	deque	Sliding window maximum, palindromes

Min stack (get the minimum in O(1))

Keep a second stack that stores the minimum so far at every depth. Pushing and popping both stacks together keeps them aligned.

```python
class MinStack:
    def __init__(self):
        self.stack = []
        self.mins = []

    def push(self, x):
        self.stack.append(x)
        self.mins.append(x if not self.mins else min(x, self.mins[-1]))

    def pop(self):
        self.mins.pop()
        return self.stack.pop()

    def top(self):
        return self.stack[-1]

    def get_min(self):
        return self.mins[-1]
```

TIME O(1) per operation

SPACE O(n)

Queue using two stacks

Push goes to an *inbox* stack. When we need to pop or peek and the *outbox* is empty, pour the whole inbox into it, which reverses the order and exposes the oldest item. Each item moves at most once, so the cost is O(1) **amortized**.

```python
class MyQueue:
    def __init__(self):
        self.inbox, self.outbox = [], []

    def push(self, x):
        self.inbox.append(x)

    def pop(self):
        self._move()
        return self.outbox.pop()

    def peek(self):
        self._move()
```

```
        return self.outbox[-1]

    def _move(self):
        if not self.outbox:
            while self.inbox:
                self.outbox.append(self.inbox.pop())
```

TIME O(1) amortized **SPACE** O(n)

6.1 Stack problems

Valid parentheses

Push every opening bracket. On a closing bracket, the top of the stack must be its matching opener. At the end the stack must be empty.

```
def is_valid(s):
    pairs = {")": "(", "]": "[", "}": "{"}
    stack = []
    for ch in s:
        if ch in pairs:
            if not stack or stack.pop() != pairs[ch]:
                return False
        else:
            stack.append(ch)
    return not stack
```

TIME O(n) **SPACE** O(n)

Evaluate reverse Polish notation

Numbers go on the stack. An operator pops two numbers, applies itself and pushes the result. Note the order: the first popped is the *right* operand. Division truncates toward zero.

```
def eval_rpn(tokens):
    stack = []
    for t in tokens:
        if t in ("+", "-", "*", "/"):
            b = stack.pop()
            a = stack.pop()
            if t == "+":
                stack.append(a + b)
            elif t == "-":
                stack.append(a - b)
            elif t == "*":
                stack.append(a * b)
            else:
                stack.append(int(a / b))     # truncate toward zero
        else:
            stack.append(int(t))
    return stack[0]
```

TIME O(n) **SPACE** O(n)

6.2 The monotonic stack pattern

Idea: keep a stack whose values are always sorted (all increasing or all decreasing). When a new item breaks the order, pop items until the order is restored; each popped item has just found *its answer* (the next greater or next smaller element). Every item is pushed once and popped once, so the whole pass is O(n) despite the inner `while` loop. Use it for 'next greater/smaller', 'how many days until warmer', and 'largest rectangle'.

Next greater element

Store indices of items still waiting for a bigger value. When x arrives, it is the answer for every waiting item smaller than it.

```python
def next_greater(nums):
    res = [-1] * len(nums)
    stack = []                              # indices; their values are decreasing
    for i, x in enumerate(nums):
        while stack and nums[stack[-1]] < x:
            res[stack.pop()] = x
        stack.append(i)
    return res
```

TIME O(n) **SPACE** O(n)

Daily temperatures

Same pattern, but store the *distance* in days to the warmer day instead of the value.

```python
def daily_temperatures(temps):
    res = [0] * len(temps)
    stack = []
    for i, t in enumerate(temps):
        while stack and temps[stack[-1]] < t:
            j = stack.pop()
            res[j] = i - j
        stack.append(i)
    return res
```

TIME O(n) **SPACE** O(n)

Largest rectangle in a histogram

Keep indices of bars with increasing heights. When a shorter bar arrives, the taller bar on top can no longer extend right: pop it and compute its rectangle. Its left edge is the new stack top. A sentinel bar of height 0 at the end flushes everything.

```python
def largest_rectangle(heights):
    stack = []                              # indices of increasing heights
    best = 0
    heights = heights + [0]                 # sentinel flushes the stack
    for i, h in enumerate(heights):
        while stack and heights[stack[-1]] > h:
            height = heights[stack.pop()]
            left = stack[-1] if stack else -1
            best = max(best, height * (i - left - 1))
        stack.append(i)
    return best
```

TIME O(n) **SPACE** O(n)

6.3 Deque problem

Sliding window maximum (monotonic deque)

The deque holds indices whose values are decreasing; the front is always the window maximum. Pop smaller values from the back (they can never be a maximum while the newer bigger value is in the window) and drop the front once it slides out of the window.

```python
from collections import deque


def max_sliding_window(nums, k):
    dq, res = deque(), []                   # dq holds indices, values decreasing
```

```
    for i, x in enumerate(nums):
        while dq and nums[dq[-1]] <= x:
            dq.pop()
        dq.append(i)
        if dq[0] <= i - k:                  # front left the window
            dq.popleft()
        if i >= k - 1:
            res.append(nums[dq[0]])
    return res
```

TIME O(n) **SPACE** O(k)

CHAPTER 7

Recursion and Backtracking

A function that calls itself, and a systematic way to explore every possibility.

7.1 Recursion in three questions

1. **Base case**: when is the problem so small that I can answer immediately? (Without it, recursion never stops.)
2. **Smaller problem**: how do I express the answer using the same function on a smaller input?
3. **Trust**: assume the recursive call returns the right answer; just combine it correctly.

Each call is pushed on the **call stack**; depth d costs O(d) space. Recursion is the natural way to think about trees, graphs, divide-and-conquer and backtracking.

Factorial and fast exponentiation

Naive power multiplies x by itself n times (O(n)). Squaring trick: `x^n = (x^(n/2))^2`, times an extra x if n is odd. Halving the exponent each call gives O(log n).

```python
def factorial(n):                           # O(n) time, O(n) stack
    return 1 if n <= 1 else n * factorial(n - 1)


def power(x, n):                            # O(log n) time and stack
    if n == 0:
        return 1
    if n < 0:
        return 1 / power(x, -n)
    half = power(x, n // 2)
    return half * half if n % 2 == 0 else half * half * x
```

TIME O(n) / O(log n) **SPACE** O(n) / O(log n)

Tower of Hanoi

To move n disks from A to C: move n-1 disks to the helper peg B, move the biggest disk, then move the n-1 disks from B onto C. The minimum number of moves is 2^n - 1.

```python
def hanoi(n, src="A", aux="B", dst="C", moves=None):
    if moves is None:
        moves = []
    if n == 0:
        return moves
    hanoi(n - 1, src, dst, aux, moves)
    moves.append((src, dst))
    hanoi(n - 1, aux, src, dst, moves)
    return moves
```

TIME O(2^n) **SPACE** O(n)

7.2 Backtracking: choose, explore, un-choose

Backtracking builds a solution one decision at a time. If a partial solution cannot succeed, abandon it immediately (**prune**) and undo the last decision to try another. It is DFS over the tree of all choices. The template is always the same:

```
def backtrack(state):
    if state is a complete solution:
        record a COPY of it          # path[:]  not  path
        return
    for choice in available choices:
        if choice is invalid:        # pruning = speed
            continue
        make the choice              # path.append(choice)
        backtrack(next state)
        undo the choice              # path.pop()
```

Classic bug: appending `path` instead of a copy `path[:]` stores a reference that later mutations change, so every saved answer ends up identical (usually empty).

Subsets (power set)

For each position decide: include or skip. Recording the path at every node of the recursion (not only at the leaves) lists all 2^n subsets. The `start` index prevents re-using earlier items and generating duplicates.

```python
def subsets(nums):
    res, path = [], []

    def backtrack(start):
        res.append(path[:])
        for i in range(start, len(nums)):
            path.append(nums[i])
            backtrack(i + 1)
            path.pop()

    backtrack(0)
    return res
```

TIME O(n * 2^n) **SPACE** O(n) stack

Permutations

At each position try every item not yet used. A `used` array marks items already in the current path.

```python
def permute(nums):
    res, path = [], []
    used = [False] * len(nums)

    def backtrack():
        if len(path) == len(nums):
            res.append(path[:])
            return
        for i in range(len(nums)):
            if used[i]:
                continue
            used[i] = True
            path.append(nums[i])
            backtrack()
            path.pop()
            used[i] = False

    backtrack()
    return res
```

TIME O(n * n!) **SPACE** O(n)

Combination sum (items can be reused)

Sorting lets us stop early (`break`) once a candidate exceeds the remaining target. Recursing with `i` instead of `i + 1` allows the same number again.

```python
def combination_sum(candidates, target):
    candidates.sort()
    res, path = [], []

    def backtrack(start, remain):
        if remain == 0:
            res.append(path[:])
            return
        for i in range(start, len(candidates)):
            if candidates[i] > remain:
                break                       # sorted: later ones are bigger too
            path.append(candidates[i])
            backtrack(i, remain - candidates[i])
            path.pop()

    backtrack(0, target)
    return res
```

TIME Exponential, bounded by target/min(candidates) depth

SPACE O(target / min) stack

Generate valid parentheses

Only two rules keep the string valid: you may add '(' while you have used fewer than n; you may add ')' only if there are more '(' than ')' so far. The rules prune all invalid branches before they are built.

```python
def generate_parentheses(n):
    res = []

    def go(cur, opened, closed):
        if len(cur) == 2 * n:
            res.append(cur)
            return
        if opened < n:
            go(cur + "(", opened + 1, closed)
        if closed < opened:
            go(cur + ")", opened, closed + 1)

    go("", 0, 0)
    return res
```

TIME O(4^n / sqrt(n)) (Catalan number)

SPACE O(n)

N-Queens

Place one queen per row. A queen is safe if its column and both diagonals are free. Diagonals are identified by `row - col` (one direction) and `row + col` (the other); three sets give O(1) safety checks.

```python
def solve_n_queens(n):
    res = []
    cols, diag1, diag2 = set(), set(), set()
    board = []                              # board[r] = column of queen in row r

    def place(r):
        if r == n:
            res.append(["." * c + "Q" + "." * (n - c - 1) for c in board])
            return
        for c in range(n):
            if c in cols or (r - c) in diag1 or (r + c) in diag2:
                continue
            cols.add(c)
            diag1.add(r - c)
            diag2.add(r + c)
            board.append(c)
            place(r + 1)
            board.pop()
            cols.remove(c)
            diag1.remove(r - c)
```

```
            diag2.remove(r + c)

    place(0)
    return res
```

TIME O(n!) worst case, far less with pruning

SPACE O(n)

Word search in a grid

DFS from every cell. Mark the current cell as visited by overwriting it, explore the four neighbours, then **restore** it (the 'un-choose' step) so other paths can use it.

```
def word_search(board, word):
    R, C = len(board), len(board[0])

    def dfs(r, c, i):
        if i == len(word):
            return True
        if r < 0 or c < 0 or r >= R or c >= C or board[r][c] != word[i]:
            return False
        tmp, board[r][c] = board[r][c], "#"  # choose (mark visited)
        found = (dfs(r + 1, c, i + 1) or dfs(r - 1, c, i + 1) or
                 dfs(r, c + 1, i + 1) or dfs(r, c - 1, i + 1))
        board[r][c] = tmp                    # un-choose
        return found

    return any(dfs(r, c, 0) for r in range(R) for c in range(C))
```

TIME O(R * C * 3^L) for word length L

SPACE O(L)

Recursion vs iteration. Anything recursive can be made iterative with an explicit stack. Prefer iteration when depth can reach 10^5 or more in Python. Prefer recursion when it makes a tree/backtracking solution dramatically clearer.

CHAPTER 8

Searching and Sorting

Find things fast, then put them in order.

8.1 Linear search vs binary search

Linear search checks items one by one: O(n), works on any list. **Binary search** works on *sorted* data: look at the middle, discard the half that cannot contain the target, repeat. Each step halves the range, so it takes O(log n): 20 steps for a million items, 30 for a billion.

Binary search (the exact-match version)

Use `lo + (hi - lo) // 2` for the midpoint (it avoids overflow in languages with fixed-size ints; harmless habit in Python). The loop condition `lo <= hi` means the range `[lo, hi]` is inclusive.

```
def binary_search(arr, target):
    lo, hi = 0, len(arr) - 1
    while lo <= hi:
        mid = lo + (hi - lo) // 2
        if arr[mid] == target:
            return mid
        if arr[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1
```

TIME O(log n) **SPACE** O(1)

Lower bound, upper bound, first and last position

The half-open version `[lo, hi)` is the most reliable template. `lower_bound` returns the first index whose value is **>= x**; `upper_bound` the first index whose value is **> x**. Together they answer 'first/last occurrence' and 'how many times does x appear' (`upper - lower`). Python's `bisect_left` and `bisect_right` are the same functions.

```
def lower_bound(arr, x):                    # first index with arr[i] >= x
    lo, hi = 0, len(arr)
    while lo < hi:
        mid = (lo + hi) // 2
        if arr[mid] < x:
            lo = mid + 1
        else:
            hi = mid
    return lo


def upper_bound(arr, x):                    # first index with arr[i] > x
    lo, hi = 0, len(arr)
    while lo < hi:
        mid = (lo + hi) // 2
        if arr[mid] <= x:
            lo = mid + 1
        else:
            hi = mid
    return lo


def first_last(arr, x):
    i = lower_bound(arr, x)
```

```
    if i == len(arr) or arr[i] != x:
        return [-1, -1]
    return [i, upper_bound(arr, x) - 1]
```

TIME O(log n) **SPACE** O(1)

Search in a rotated sorted array

After rotation, at least one half around `mid` is still sorted. Decide which half is sorted, check whether the target lies inside it, and discard the other half.

```
def search_rotated(nums, target):
    lo, hi = 0, len(nums) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if nums[mid] == target:
            return mid
        if nums[lo] <= nums[mid]:            # left half is sorted
            if nums[lo] <= target < nums[mid]:
                hi = mid - 1
            else:
                lo = mid + 1
        else:                                # right half is sorted
            if nums[mid] < target <= nums[hi]:
                lo = mid + 1
            else:
                hi = mid - 1
    return -1
```

TIME O(log n) **SPACE** O(1)

8.2 Binary search on the answer

The most powerful generalization. You do not need a sorted array; you need a *yes/no question that flips exactly once* as the answer grows (for example 'can I finish with speed k?' is no, no, no, yes, yes, yes). Binary search finds the flip point. Whenever a problem says 'minimize the maximum' or 'maximize the minimum', think of it.

Koko eating bananas (minimum speed)

Speed k works if total hours `sum(ceil(pile / k))` is at most h. Feasibility is monotonic: faster is never worse. Search k between 1 and the largest pile.

```
def min_eating_speed(piles, h):
    lo, hi = 1, max(piles)
    while lo < hi:
        mid = (lo + hi) // 2
        hours = sum((p + mid - 1) // mid for p in piles)   # ceil without floats
        if hours <= h:
            hi = mid                         # mid works; try slower
        else:
            lo = mid + 1                     # too slow
    return lo
```

TIME O(n log(max pile)) **SPACE** O(1)

Integer square root without sqrt()

Find the largest number whose square is at most x.

```
def int_sqrt(x):
    lo, hi, ans = 0, x, 0
    while lo <= hi:
        mid = (lo + hi) // 2
        if mid * mid <= x:
            ans = mid
```

```
            lo = mid + 1
        else:
            hi = mid - 1
    return ans
```

TIME O(log x) **SPACE** O(1)

8.3 Sorting algorithms

Sorting appears everywhere: it enables binary search, two pointers, grouping and greedy choices. Know the simple O(n^2) sorts for understanding, and the O(n log n) sorts for use.

Bubble, selection and insertion sort (O(n^2))

Bubble: repeatedly swap adjacent out-of-order pairs; the largest value 'bubbles' to the end each pass. **Selection**: repeatedly select the minimum of the rest and put it next. **Insertion**: grow a sorted prefix by inserting each new item into its place; excellent on nearly-sorted data (O(n) best case) and the algorithm inside real-world hybrid sorts for small arrays.

```
def bubble_sort(a):
    n = len(a)
    for i in range(n):
        swapped = False
        for j in range(n - 1 - i):
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                swapped = True
        if not swapped:                    # already sorted: stop early
            break
    return a


def selection_sort(a):
    n = len(a)
    for i in range(n):
        m = i
        for j in range(i + 1, n):
            if a[j] < a[m]:
                m = j
        a[i], a[m] = a[m], a[i]
    return a


def insertion_sort(a):
    for i in range(1, len(a)):
        key, j = a[i], i - 1
        while j >= 0 and a[j] > key:
            a[j + 1] = a[j]                # shift bigger items right
            j -= 1
        a[j + 1] = key
    return a
```

TIME O(n^2) (insertion best case O(n)) **SPACE** O(1)

Merge sort (divide and conquer, stable)

Split in half, sort each half recursively, then **merge** two sorted halves by repeatedly taking the smaller front item. Always O(n log n); stable (equal items keep their order) thanks to <=; costs O(n) extra memory.

```
def merge_sort(a):
    if len(a) <= 1:
        return a
    mid = len(a) // 2
    left, right = merge_sort(a[:mid]), merge_sort(a[mid:])
    out, i, j = [], 0, 0
```

```
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:              # <= keeps the sort stable
            out.append(left[i])
            i += 1
        else:
            out.append(right[j])
            j += 1
    out.extend(left[i:])
    out.extend(right[j:])
    return out
```

TIME O(n log n) **SPACE** O(n)

Quick sort (partition, in place)

Pick a pivot, **partition** so smaller items go left and larger go right (the pivot lands in its final position), then sort both sides. A *random* pivot makes the O(n^2) worst case (e.g., sorted input with a fixed pivot) vanishingly unlikely. Average O(n log n), in place, usually the fastest in practice. Lomuto partition shown here is the easiest to write; heavy duplicates slow it down (use 3-way partition then).

```
import random


def quick_sort(a, lo=0, hi=None):
    if hi is None:
        hi = len(a) - 1
    if lo < hi:
        p = _partition(a, lo, hi)
        quick_sort(a, lo, p - 1)
        quick_sort(a, p + 1, hi)
    return a


def _partition(a, lo, hi):
    r = random.randint(lo, hi)               # random pivot
    a[r], a[hi] = a[hi], a[r]
    pivot, i = a[hi], lo
    for j in range(lo, hi):
        if a[j] < pivot:
            a[i], a[j] = a[j], a[i]
            i += 1
    a[i], a[hi] = a[hi], a[i]                # pivot to its final position
    return i
```

TIME O(n log n) average, O(n^2) worst **SPACE** O(log n) stack

Heap sort (in place, guaranteed O(n log n))

Build a max-heap inside the array (O(n)), then repeatedly swap the maximum to the end and restore the heap on the shrunken front part. The Heaps chapter explains `sift_down` in detail.

```
def heap_sort(a):
    n = len(a)

    def sift_down(i, size):
        while True:
            big, l, r = i, 2 * i + 1, 2 * i + 2
            if l < size and a[l] > a[big]:
                big = l
            if r < size and a[r] > a[big]:
                big = r
            if big == i:
                return
            a[i], a[big] = a[big], a[i]
            i = big
```

```
    for i in range(n // 2 - 1, -1, -1):      # build max-heap in O(n)
        sift_down(i, n)
    for end in range(n - 1, 0, -1):          # move max to the back
        a[0], a[end] = a[end], a[0]
        sift_down(0, end)
    return a
```

TIME O(n log n) **SPACE** O(1)

Counting sort (no comparisons)

When values lie in a small range, count how many times each value occurs and rebuild the array. Beats the O(n log n) comparison-sort limit because it never compares two items. Radix sort applies this digit by digit.

```
def counting_sort(a):
    if not a:
        return a
    lo, hi = min(a), max(a)
    count = [0] * (hi - lo + 1)
    for x in a:
        count[x - lo] += 1
    out = []
    for i, c in enumerate(count):
        out.extend([i + lo] * c)
    return out
```

TIME O(n + k) for value range k **SPACE** O(k)

Algorithm	Best	Average	Worst	Space	Stable?	Notes
Bubble	O(n)	O(n^2)	O(n^2)	O(1)	Yes	Teaching only
Selection	O(n^2)	O(n^2)	O(n^2)	O(1)	No	Fewest swaps
Insertion	O(n)	O(n^2)	O(n^2)	O(1)	Yes	Great for small/nearly sorted
Merge	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes	Predictable; linked lists
Quick	O(n log n)	O(n log n)	O(n^2)	O(log n)	No	Fastest in practice
Heap	O(n log n)	O(n log n)	O(n log n)	O(1)	No	In place, no bad case
Counting	O(n + k)	O(n + k)	O(n + k)	O(k)	Yes	Small integer range
Python sorted()	O(n)	O(n log n)	O(n log n)	O(n)	Yes	Timsort: merge + insertion

In real code, just call `sorted()` **or** `list.sort()` (use `key=` to sort by a property). Implement sorts yourself to learn the ideas and for interviews, not for production.

Quickselect: k-th smallest in average O(n)

Use the same partition as quicksort but recurse into *one* side only: the side that contains position k. Much faster than sorting when you need just one rank. (Reuses `_partition` from the quick sort listing above.)

```
def kth_smallest(a, k):                      # k is 1-indexed
    a = a[:]
    lo, hi = 0, len(a) - 1
    while True:
        p = _partition(a, lo, hi)
        if p == k - 1:
            return a[p]
        if p < k - 1:
            lo = p + 1
        else:
            hi = p - 1
```

TIME O(n) average, O(n^2) worst **SPACE** O(n) copy

CHAPTER 9

Trees and Binary Search Trees

Hierarchical data: file systems, HTML, org charts, databases.

9.1 Vocabulary

- **Root**: the top node. **Leaf**: a node with no children. **Parent / child / sibling**: as in a family tree.
- **Depth** of a node: edges from the root. **Height** of a tree: edges (or nodes) on the longest root-to-leaf path.
- **Binary tree**: each node has at most two children (left, right). **Balanced**: heights of the two subtrees differ by at most 1 at every node.
- **Binary Search Tree (BST)**: for every node, everything in its left subtree is smaller and everything in its right subtree is larger. In-order traversal of a BST gives sorted order.

```
          8              In-order  (L, Root, R): 1 3 4 6 7 8 10 14
        /   \                Pre-order (Root, L, R): 8 3 1 6 4 7 10 14
       3     10              Post-order (L, R, Root): 1 4 7 6 3 14 10 8
      / \      \             Level-order (BFS)      : 8 | 3 10 | 1 6 14 | 4 7
     1   6     14
        / \
       4   7
```

TreeNode and a test helper

`build_tree` builds a tree from a level-order list in which `None` marks a missing child (the format used by most online judges). Later listings reuse it.

```python
from collections import deque


class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right


def build_tree(vals):                       # level-order list with None
    if not vals or vals[0] is None:
        return None
    root = TreeNode(vals[0])
    q = deque([root])
    i = 1
    while q and i < len(vals):
        node = q.popleft()
        if i < len(vals) and vals[i] is not None:
            node.left = TreeNode(vals[i])
            q.append(node.left)
        i += 1
        if i < len(vals) and vals[i] is not None:
            node.right = TreeNode(vals[i])
            q.append(node.right)
        i += 1
    return root
```

TIME O(n) **SPACE** O(n)

9.2 Traversals

Traversal means visiting every node exactly once. **Depth-first** (DFS) goes deep before wide and has three orders (pre, in, post). **Breadth-first** (BFS, level order) visits level by level using a queue. All are O(n) time; DFS uses O(h) stack where h is the height (O(log n) balanced, O(n) skewed); BFS uses O(w) for the widest level.

Recursive DFS: in-order, pre-order, post-order

The three orders differ only in *when* you record the node: before the children (pre), between them (in) or after them (post). Pre-order copies a tree, in-order sorts a BST, post-order deletes a tree or computes things from the bottom up.

```python
def inorder(root):
    res = []

    def dfs(node):
        if node:
            dfs(node.left)
            res.append(node.val)            # between the children
            dfs(node.right)

    dfs(root)
    return res


def preorder(root):
    res = []

    def dfs(node):
        if node:
            res.append(node.val)            # before the children
            dfs(node.left)
            dfs(node.right)

    dfs(root)
    return res


def postorder(root):
    res = []

    def dfs(node):
        if node:
            dfs(node.left)
            dfs(node.right)
            res.append(node.val)            # after the children

    dfs(root)
    return res
```

TIME O(n) **SPACE** O(h)

Iterative DFS (explicit stack) and level order BFS

Iterative versions avoid Python's recursion limit. In-order: go as far left as possible, pop, record, then move right. Post-order trick: do a modified pre-order (root, right, left) and reverse the result. BFS processes one whole level per outer iteration using `len(q)`.

```python
def inorder_iter(root):
    res, stack, cur = [], [], root
    while cur or stack:
        while cur:
            stack.append(cur)
            cur = cur.left
        cur = stack.pop()
        res.append(cur.val)
        cur = cur.right
```

```
    return res


def preorder_iter(root):
    if not root:
        return []
    res, stack = [], [root]
    while stack:
        node = stack.pop()
        res.append(node.val)
        if node.right:
            stack.append(node.right)
        if node.left:
            stack.append(node.left)
    return res


def postorder_iter(root):
    if not root:
        return []
    res, stack = [], [root]
    while stack:
        node = stack.pop()
        res.append(node.val)
        if node.left:
            stack.append(node.left)
        if node.right:
            stack.append(node.right)
    return res[::-1]


def level_order(root):
    if not root:
        return []
    res, q = [], deque([root])
    while q:
        level = []
        for _ in range(len(q)):              # exactly one level
            node = q.popleft()
            level.append(node.val)
            if node.left:
                q.append(node.left)
            if node.right:
                q.append(node.right)
        res.append(level)
    return res
```

TIME O(n) **SPACE** O(h) DFS, O(width) BFS

9.3 Bottom-up tree recursion

Most tree problems follow one pattern: **solve the left subtree, solve the right subtree, combine**. Decide what each call *returns* to its parent (often a height or a gain) and what it records on the side (often the global best, via `nonlocal`).

Height, diameter and balance check

Height = 1 + max of children. **Diameter** (longest path between any two nodes) = max over nodes of left height + right height, computed in the same pass. **Balance**: return -1 as a signal for 'unbalanced' so we stop early, all in one traversal instead of recomputing heights.

```
def max_depth(root):
    return 0 if not root else 1 + max(max_depth(root.left), max_depth(root.right))


def diameter(root):                          # in edges
```

```
    best = 0

    def depth(node):
        nonlocal best
        if not node:
            return 0
        l, r = depth(node.left), depth(node.right)
        best = max(best, l + r)
        return 1 + max(l, r)

    depth(root)
    return best


def is_balanced(root):
    def check(node):                        # returns height, or -1 if unbalanced
        if not node:
            return 0
        l = check(node.left)
        if l == -1:
            return -1
        r = check(node.right)
        if r == -1 or abs(l - r) > 1:
            return -1
        return 1 + max(l, r)

    return check(root) != -1
```

TIME O(n) **SPACE** O(h)

Invert, symmetric and path-sum

Invert: swap children at every node. **Symmetric**: the tree equals its mirror image; compare left subtree to right subtree with crossed children. **Path sum**: subtract the node value on the way down; success is a leaf where the remainder is exactly 0.

```
def invert_tree(root):
    if root:
        root.left, root.right = invert_tree(root.right), invert_tree(root.left)
    return root


def is_symmetric(root):
    def mirror(a, b):
        if not a and not b:
            return True
        if not a or not b or a.val != b.val:
            return False
        return mirror(a.left, b.right) and mirror(a.right, b.left)

    return mirror(root.left, root.right) if root else True


def has_path_sum(root, target):
    if not root:
        return False
    if not root.left and not root.right:
        return target == root.val
    return (has_path_sum(root.left, target - root.val) or
            has_path_sum(root.right, target - root.val))
```

TIME O(n) **SPACE** O(h)

Maximum path sum (any node to any node)

For each node, the best path that *bends* there is `node + left_gain + right_gain` (ignore negative gains). But the value returned upward may use only one branch, because a path cannot fork when it continues to the parent.

```python
def max_path_sum(root):
    best = float("-inf")

    def gain(node):
        nonlocal best
        if not node:
            return 0
        l = max(gain(node.left), 0)          # ignore negative branches
        r = max(gain(node.right), 0)
        best = max(best, node.val + l + r)   # path that bends at this node
        return node.val + max(l, r)          # path that continues upward

    gain(root)
    return best
```

TIME O(n) **SPACE** O(h)

Lowest common ancestor (binary tree)

If the current node is p or q, return it. Otherwise ask both subtrees. If both sides report a find, the current node is where the paths split: it is the LCA. If only one side reports, pass that result up.

```python
def lca(root, p, q):                         # p and q are node objects
    if not root or root is p or root is q:
        return root
    left = lca(root.left, p, q)
    right = lca(root.right, p, q)
    return root if left and right else (left or right)
```

TIME O(n) **SPACE** O(h)

Build a tree from pre-order and in-order lists

The first pre-order item is the root. Find it in the in-order list: everything left of it is the left subtree, everything right is the right subtree. A hash map of in-order positions makes the lookup O(1); an iterator over pre-order hands out roots in the correct order.

```python
def build_from_pre_in(preorder, inorder):
    idx = {v: i for i, v in enumerate(inorder)}
    pre = iter(preorder)

    def build(lo, hi):
        if lo > hi:
            return None
        root = TreeNode(next(pre))
        mid = idx[root.val]
        root.left = build(lo, mid - 1)
        root.right = build(mid + 1, hi)
        return root

    return build(0, len(inorder) - 1)
```

TIME O(n) **SPACE** O(n)

9.4 Binary Search Trees

Because of the ordering rule, a BST search discards half the tree at each step: O(h). A **balanced** BST has h = log n, so search/insert/delete are O(log n); a *skewed* BST (inserting sorted data) degenerates into a linked list with h = n. Self-balancing trees (AVL, Red-Black) rotate nodes to guarantee O(log n); Python's `sortedcontainers` and C++'s `std::map` serve this need.

BST insert, search and delete

Delete has three cases: a leaf (just remove), one child (replace the node by its child), two children (copy the *in-order successor*, the smallest value of the right subtree, into the node, then delete that successor).

```python
def bst_insert(root, val):
    if not root:
        return TreeNode(val)
    if val < root.val:
        root.left = bst_insert(root.left, val)
    elif val > root.val:
        root.right = bst_insert(root.right, val)
    return root


def bst_search(root, val):
    while root and root.val != val:
        root = root.left if val < root.val else root.right
    return root


def bst_delete(root, key):
    if not root:
        return None
    if key < root.val:
        root.left = bst_delete(root.left, key)
    elif key > root.val:
        root.right = bst_delete(root.right, key)
    else:
        if not root.left:
            return root.right
        if not root.right:
            return root.left
        succ = root.right                    # smallest in the right subtree
        while succ.left:
            succ = succ.left
        root.val = succ.val
        root.right = bst_delete(root.right, succ.val)
    return root
```

TIME O(h): O(log n) balanced, O(n) skewed **SPACE** O(h)

Validate a BST and find the k-th smallest

Checking only a node against its direct children is a classic mistake. Every node must lie inside a *(low, high)* window inherited from all its ancestors. The k-th smallest is simply the k-th item of an in-order walk; stop as soon as you reach it.

```python
def is_valid_bst(root, lo=float("-inf"), hi=float("inf")):
    if not root:
        return True
    if not (lo < root.val < hi):
        return False
    return (is_valid_bst(root.left, lo, root.val) and
            is_valid_bst(root.right, root.val, hi))


def kth_smallest_bst(root, k):
    stack, cur = [], root
    while cur or stack:
        while cur:
            stack.append(cur)
            cur = cur.left
        cur = stack.pop()
        k -= 1
        if k == 0:
            return cur.val
        cur = cur.right
    return None
```

TIME O(n) / O(h + k) **SPACE** O(h)

CHAPTER 10

Heaps and Priority Queues

Always know the smallest (or largest) item in O(1); update it in O(log n).

A **heap** is a complete binary tree stored in an array where every parent is <= its children (a *min-heap*). The smallest item is always at index 0. Python's `heapq` provides a min-heap; for a max-heap, store negated values. A heap is the engine behind a **priority queue**: 'give me the most important item next'.

```
Min-heap as a tree:          Same heap as an array:
        1                     index:  0  1  2  3  4  5
      /   \                   value: [1, 3, 2, 7, 4, 5]
     3     2
    / \   /                   parent of i  = (i - 1) // 2
   7   4 5                    children of i = 2i + 1 and 2i + 2
```

Operation	Cost	How
peek min	O(1)	heap[0]
push	O(log n)	Add at the end, sift up while smaller than parent
pop min	O(log n)	Swap root with last, remove last, sift down
build heap from n items	O(n)	heapify: sift down from the last parent upward
search arbitrary item	O(n)	Heaps are not ordered for searching

Implement a min-heap from scratch

Sift up after a push (the new item floats up while smaller than its parent). **Sift down** after a pop (the item moved to the root sinks toward the smaller child until order is restored).

```python
class MinHeap:
    def __init__(self):
        self.a = []

    def push(self, x):                          # O(log n)
        self.a.append(x)
        i = len(self.a) - 1
        while i > 0:
            parent = (i - 1) // 2
            if self.a[parent] <= self.a[i]:
                break
            self.a[parent], self.a[i] = self.a[i], self.a[parent]
            i = parent

    def pop(self):                              # O(log n)
        a = self.a
        a[0], a[-1] = a[-1], a[0]
        smallest = a.pop()
        i, n = 0, len(a)
        while True:
            l, r, small = 2 * i + 1, 2 * i + 2, i
            if l < n and a[l] < a[small]:
                small = l
            if r < n and a[r] < a[small]:
                small = r
            if small == i:
                break
            a[i], a[small] = a[small], a[i]
            i = small
        return smallest
```

```
    def peek(self):
        return self.a[0]

    def __len__(self):
        return len(self.a)
```

TIME push/pop O(log n), peek O(1) **SPACE** O(n)

10.1 Top-K problems

Rule: to keep the K *largest* items use a **min-heap of size K** (the smallest of the best K sits on top, ready to be evicted). To keep the K smallest, use a max-heap of size K. This costs O(n log K), better than sorting everything when K is small.

K-th largest element

Push each number; whenever the heap grows beyond K, evict the smallest. The root of the final heap is the K-th largest.

```
import heapq


def kth_largest(nums, k):
    heap = []                                   # min-heap holding the k largest so far
    for x in nums:
        heapq.heappush(heap, x)
        if len(heap) > k:
            heapq.heappop(heap)
    return heap[0]
```

TIME O(n log k) **SPACE** O(k)

Top K frequent elements

Count frequencies with a `Counter`, then pick the K largest counts. The heap version is O(n log K). The **bucket sort** version is O(n): bucket index = frequency, because no frequency can exceed n.

```
from collections import Counter
import heapq


def top_k_frequent(nums, k):
    freq = Counter(nums)
    return [x for x, _ in heapq.nlargest(k, freq.items(), key=lambda p: p[1])]


def top_k_frequent_bucket(nums, k):
    freq = Counter(nums)
    buckets = [[] for _ in range(len(nums) + 1)]   # buckets[c] = values seen c times
    for x, c in freq.items():
        buckets[c].append(x)
    res = []
    for c in range(len(buckets) - 1, 0, -1):
        for x in buckets[c]:
            res.append(x)
            if len(res) == k:
                return res
    return res
```

TIME O(n log k) / O(n) **SPACE** O(n)

K closest points to the origin

Compare squared distances (no square root needed). `nsmallest` uses a heap internally.

```python
import heapq


def k_closest(points, k):
    return heapq.nsmallest(k, points, key=lambda p: p[0] ** 2 + p[1] ** 2)
```

TIME O(n log k) **SPACE** O(k)

10.2 Merging and streaming

Merge K sorted lists

The heap always holds the current front item of each list. Pop the smallest, then push the next item from the same list. Tuples `(value, list_id, position)` compare by value first. Same idea merges K sorted linked lists or K sorted files too big for memory.

```python
import heapq


def merge_k_sorted(lists):
    heap = [(lst[0], i, 0) for i, lst in enumerate(lists) if lst]
    heapq.heapify(heap)
    out = []
    while heap:
        val, i, j = heapq.heappop(heap)
        out.append(val)
        if j + 1 < len(lists[i]):
            heapq.heappush(heap, (lists[i][j + 1], i, j + 1))
    return out
```

TIME O(N log k) for N total items **SPACE** O(k)

Median of a data stream (two heaps)

Keep the lower half in a *max-heap* and the upper half in a *min-heap*, sizes differing by at most one. The median is then at the heap tops. Each insert costs O(log n); each median query is O(1).

```python
import heapq


class MedianFinder:
    def __init__(self):
        self.small = []                     # max-heap (negated): lower half
        self.large = []                     # min-heap: upper half

    def add(self, x):
        heapq.heappush(self.small, -x)
        heapq.heappush(self.large, -heapq.heappop(self.small))
        if len(self.large) > len(self.small):
            heapq.heappush(self.small, -heapq.heappop(self.large))

    def median(self):
        if len(self.small) > len(self.large):
            return -self.small[0]
        return (-self.small[0] + self.large[0]) / 2
```

TIME add O(log n), median O(1) **SPACE** O(n)

When you see: 'K largest/smallest', 'next smallest repeatedly', 'merge K sorted', 'schedule by priority', 'running median', or Dijkstra's shortest path. Think heap.

CHAPTER 11

Graphs

Networks of things and the connections between them: maps, social networks, dependencies.

11.1 Vocabulary and representation

- A graph has **vertices** (nodes) V and **edges** E. Edges can be **directed** (one-way) or **undirected**, and **weighted** (carry a cost) or unweighted.
- **Path**: a sequence of connected vertices. **Cycle**: a path that returns to its start. **Connected**: every vertex reachable from every other. **DAG**: directed acyclic graph.
- A **tree** is a connected undirected graph with no cycles (n vertices, n-1 edges).

Representation	Space	Edge check (u, v)	List neighbours of u	Best for
Adjacency list	O(V + E)	O(deg(u))	O(deg(u))	Sparse graphs (most real graphs)
Adjacency matrix	O(V^2)	O(1)	O(V)	Dense graphs, Floyd-Warshall
Edge list	O(E)	O(E)	O(E)	Kruskal, Bellman-Ford

Build an adjacency list

Nodes are numbered 0 to n-1. The same helper is reused by every algorithm in this chapter. For undirected graphs each edge is stored in both directions.

```
from collections import deque
import heapq


def build_graph(n, edges, directed=False):
    g = [[] for _ in range(n)]
    for u, v in edges:
        g[u].append(v)
        if not directed:
            g[v].append(u)
    return g
```

TIME O(V + E) **SPACE** O(V + E)

11.2 BFS and DFS

BFS (breadth-first) explores in rings around the start using a **queue**. In an unweighted graph it finds the **shortest path** in number of edges. **DFS (depth-first)** dives as deep as possible using a **stack** (or recursion) and backtracks; it is the tool for cycles, components, topological order and exhaustive search. Both visit each vertex and edge once: O(V + E). Always keep a `seen` set so cycles do not loop forever.

BFS and DFS (recursive and iterative)

The only structural difference: BFS takes from the *front* of a queue, DFS takes from the *back* of a stack. Mark nodes as seen when you *add* them to the queue in BFS to prevent duplicates.

```
def bfs(g, start):
    seen = {start}
    q = deque([start])
    order = []
    while q:
        u = q.popleft()
        order.append(u)
        for v in g[u]:
```

```
            if v not in seen:
                seen.add(v)
                q.append(v)
    return order


def dfs_recursive(g, u, seen=None, order=None):
    if seen is None:
        seen, order = set(), []
    seen.add(u)
    order.append(u)
    for v in g[u]:
        if v not in seen:
            dfs_recursive(g, v, seen, order)
    return order


def dfs_iterative(g, start):
    seen, stack, order = set(), [start], []
    while stack:
        u = stack.pop()
        if u in seen:
            continue
        seen.add(u)
        order.append(u)
        for v in reversed(g[u]):            # reversed: visit in listed order
            if v not in seen:
                stack.append(v)
    return order
```

TIME O(V + E) **SPACE** O(V)

Connected components and number of islands

Start a traversal from every unvisited node: each start is a new component. A grid is just an implicit graph whose neighbours are up, down, left and right. 'Sinking' land cells (overwriting with '0') replaces the seen set.

```
def count_components(n, edges):
    g = build_graph(n, edges)
    seen, count = set(), 0
    for s in range(n):
        if s not in seen:
            count += 1
            seen.add(s)
            stack = [s]
            while stack:
                u = stack.pop()
                for v in g[u]:
                    if v not in seen:
                        seen.add(v)
                        stack.append(v)
    return count


def num_islands(grid):
    R, C = len(grid), len(grid[0])
    count = 0
    for r in range(R):
        for c in range(C):
            if grid[r][c] == "1":
                count += 1
                grid[r][c] = "0"
                stack = [(r, c)]
                while stack:
                    x, y = stack.pop()
                    for dx, dy in ((1, 0), (-1, 0), (0, 1), (0, -1)):
                        nx, ny = x + dx, y + dy
                        if 0 <= nx < R and 0 <= ny < C and grid[nx][ny] == "1":
```

```
                            grid[nx][ny] = "0"
                            stack.append((nx, ny))
    return count
```

TIME O(V + E) / O(R * C) **SPACE** O(V) / O(R * C)

Shortest path in a grid (BFS)

BFS reaches cells in order of distance, so the first time we pop the target its distance is the minimum. Store the distance together with each queue entry.

```
def shortest_path_grid(grid, start, end):    # 0 = open cell, 1 = wall
    R, C = len(grid), len(grid[0])
    q = deque([(start[0], start[1], 0)])
    seen = {start}
    while q:
        r, c, d = q.popleft()
        if (r, c) == end:
            return d
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nr, nc = r + dr, c + dc
            if (0 <= nr < R and 0 <= nc < C and grid[nr][nc] == 0
                    and (nr, nc) not in seen):
                seen.add((nr, nc))
                q.append((nr, nc, d + 1))
    return -1
```

TIME O(R * C) **SPACE** O(R * C)

11.3 Cycles, ordering and bipartite graphs

Detect a cycle: undirected and directed

Undirected: while doing DFS, meeting an already-seen vertex that is *not your parent* means a cycle. **Directed**: use three colours: 0 = unvisited, 1 = on the current recursion path, 2 = finished. Meeting a vertex with colour 1 (a back edge) means a cycle; meeting colour 2 is harmless.

```
def has_cycle_undirected(n, edges):
    g = build_graph(n, edges)
    seen = set()

    def dfs(u, parent):
        seen.add(u)
        for v in g[u]:
            if v == parent:
                continue
            if v in seen or dfs(v, u):
                return True
        return False

    return any(s not in seen and dfs(s, -1) for s in range(n))


def has_cycle_directed(n, edges):
    g = build_graph(n, edges, directed=True)
    state = [0] * n                          # 0 new, 1 on current path, 2 done

    def dfs(u):
        state[u] = 1
        for v in g[u]:
            if state[v] == 1 or (state[v] == 0 and dfs(v)):
                return True
        state[u] = 2
        return False

    return any(state[s] == 0 and dfs(s) for s in range(n))
```

TIME O(V + E) **SPACE** O(V)

Topological sort (Kahn's algorithm) and course schedule

A topological order lists vertices so that every edge goes from earlier to later (build order, course prerequisites). Repeatedly take a vertex with **in-degree 0** (no unmet prerequisites), output it and decrement its neighbours. If we cannot output all vertices, a cycle exists.

```python
def topological_sort(n, edges):              # returns [] if there is a cycle
    g = build_graph(n, edges, directed=True)
    indeg = [0] * n
    for u, v in edges:
        indeg[v] += 1
    q = deque(i for i in range(n) if indeg[i] == 0)
    order = []
    while q:
        u = q.popleft()
        order.append(u)
        for v in g[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                q.append(v)
    return order if len(order) == n else []


def can_finish(num_courses, prerequisites):  # pairs [course, prerequisite]
    edges = [(b, a) for a, b in prerequisites]
    return len(topological_sort(num_courses, edges)) == num_courses
```

TIME O(V + E) **SPACE** O(V)

Is the graph bipartite? (two-colouring)

Try to colour vertices with two colours so that every edge connects different colours. BFS assigns each neighbour the opposite colour; a conflict means an odd cycle, so the graph is not bipartite. Handy for 'split into two groups with no conflicts'.

```python
def is_bipartite(n, edges):
    g = build_graph(n, edges)
    color = [-1] * n
    for s in range(n):
        if color[s] != -1:
            continue
        color[s] = 0
        q = deque([s])
        while q:
            u = q.popleft()
            for v in g[u]:
                if color[v] == -1:
                    color[v] = 1 - color[u]
                    q.append(v)
                elif color[v] == color[u]:
                    return False
    return True
```

TIME O(V + E) **SPACE** O(V)

11.4 Shortest paths in weighted graphs

Algorithm	Handles	Time	Use when
BFS	Unweighted edges	O(V + E)	All edges cost the same
Dijkstra	Non-negative weights	O((V + E) log V)	Single source, typical road/network problems
Bellman-Ford	Negative weights, detects negative cycles	O(V * E)	Negative edges or currency arbitrage
Floyd-Warshall	All pairs, negative weights (no negative cycle)	O(V^3)	Small graphs (V up to a few hundred)

Dijkstra's algorithm

Greedy: always finalize the unvisited vertex with the smallest known distance, then **relax** its edges (check whether going through it gives a shorter route). A min-heap supplies that vertex. A popped entry that is worse than the stored distance is stale and is skipped. Fails with negative weights.

```python
def dijkstra(n, edges, src):                # edges: (u, v, w), directed, w >= 0
    g = [[] for _ in range(n)]
    for u, v, w in edges:
        g[u].append((v, w))
    dist = [float("inf")] * n
    dist[src] = 0
    heap = [(0, src)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:                     # stale entry
            continue
        for v, w in g[u]:
            if d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(heap, (dist[v], v))
    return dist
```

TIME O((V + E) log V) **SPACE** O(V + E)

Bellman-Ford (negative edges and negative cycles)

A shortest path uses at most V-1 edges, so relaxing *every* edge V-1 times is enough. If a further pass still improves something, there is a negative cycle and shortest paths are undefined (we return None).

```python
def bellman_ford(n, edges, src):            # edges: (u, v, w)
    dist = [float("inf")] * n
    dist[src] = 0
    for _ in range(n - 1):
        changed = False
        for u, v, w in edges:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                changed = True
        if not changed:
            break
    for u, v, w in edges:                   # one more pass: improvement = negative cycle
        if dist[u] + w < dist[v]:
            return None
    return dist
```

TIME O(V * E) **SPACE** O(V)

Floyd-Warshall (all pairs shortest paths)

Dynamic programming over intermediate vertices: `d[i][j] = min(d[i][j], d[i][k] + d[k][j])`. After processing vertex k, `d[i][j]` is the best path using only vertices 0..k as stepping stones. The loop order `k, i, j` matters.

```python
def floyd_warshall(n, edges):                # edges: (u, v, w), directed
    INF = float("inf")
    d = [[0 if i == j else INF for j in range(n)] for i in range(n)]
    for u, v, w in edges:
        d[u][v] = min(d[u][v], w)
    for k in range(n):
        for i in range(n):
            for j in range(n):
                if d[i][k] + d[k][j] < d[i][j]:
                    d[i][j] = d[i][k] + d[k][j]
    return d
```

TIME O(V^3) **SPACE** O(V^2)

11.5 Union-Find and minimum spanning trees

Union-Find (Disjoint Set Union) keeps track of groups of connected items: `find(x)` returns the group leader, `union(a, b)` merges two groups. With **path compression** (flatten the tree during find) and **union by rank** (attach the shorter tree under the taller), each operation is nearly O(1) (inverse Ackermann function). A **minimum spanning tree (MST)** connects all vertices with the cheapest total edge weight and no cycle.

Union-Find (DSU)

Also solves 'are these two connected?', 'how many groups?' and offline cycle detection in undirected graphs: if `union` returns False, the edge would close a cycle.

```python
class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n
        self.components = n

    def find(self, x):                       # with path compression
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a, b):                   # by rank; False if already joined
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False
        if self.rank[ra] < self.rank[rb]:
            ra, rb = rb, ra
        self.parent[rb] = ra
        if self.rank[ra] == self.rank[rb]:
            self.rank[ra] += 1
        self.components -= 1
        return True
```

TIME Almost O(1) per operation **SPACE** O(n)

Kruskal's and Prim's minimum spanning tree

Kruskal: sort edges by weight; take an edge if it joins two different groups (checked with DSU). Good for sparse graphs and edge lists. **Prim**: grow one tree from a start vertex, always adding the cheapest edge that leaves the tree (a heap supplies it). Good for dense graphs. Both return the same total weight.

```python
def kruskal(n, edges):                       # edges: (w, u, v)
    dsu, total, used = DSU(n), 0, []
    for w, u, v in sorted(edges):
        if dsu.union(u, v):
            total += w
            used.append((u, v, w))
    return total, used
```

```python
def prim(n, edges):                          # edges: (u, v, w); graph must be connected
    g = [[] for _ in range(n)]
    for u, v, w in edges:
        g[u].append((v, w))
        g[v].append((u, w))
    seen, heap, total = set(), [(0, 0)], 0
    while heap and len(seen) < n:
        w, u = heapq.heappop(heap)
        if u in seen:
            continue
        seen.add(u)
        total += w
        for v, wt in g[u]:
            if v not in seen:
                heapq.heappush(heap, (wt, v))
    return total
```

TIME Kruskal O(E log E), Prim O(E log V) **SPACE** O(V + E)

Strongly connected components (Kosaraju)

In a directed graph, a strongly connected component (SCC) is a maximal set of vertices that can all reach each other. Pass 1: DFS on the graph and record vertices by finishing time. Pass 2: DFS on the *reversed* graph in decreasing finishing time; each tree found is one SCC.

```python
def kosaraju_scc(n, edges):
    g = build_graph(n, edges, directed=True)
    rg = build_graph(n, [(v, u) for u, v in edges], directed=True)
    seen, order = [False] * n, []

    def dfs1(u):
        seen[u] = True
        for v in g[u]:
            if not seen[v]:
                dfs1(v)
        order.append(u)                      # record by finishing time

    for s in range(n):
        if not seen[s]:
            dfs1(s)

    comp, comps = [-1] * n, []

    def dfs2(u):
        comp[u] = len(comps) - 1
        comps[-1].append(u)
        for v in rg[u]:
            if comp[v] == -1:
                dfs2(v)

    for s in reversed(order):
        if comp[s] == -1:
            comps.append([])
            dfs2(s)
    return comps
```

TIME O(V + E) **SPACE** O(V + E)

Which algorithm? Fewest edges in an unweighted graph: BFS. Weighted, no negatives: Dijkstra. Negative edges: Bellman-Ford. All pairs on a small graph: Floyd-Warshall. Dependencies or ordering: topological sort. Groups or connectivity: DSU or DFS. Cheapest way to connect everything: Kruskal or Prim.

CHAPTER 12

Dynamic Programming

Solve each subproblem once, remember the answer, never repeat work.

12.1 What is dynamic programming?

Dynamic programming (DP) is **recursion plus memory**. It applies when a problem has two properties:

- **Overlapping subproblems**: the same smaller question is asked many times (naive Fibonacci asks for fib(3) over and over).
- **Optimal substructure**: the best answer to the whole is built from the best answers to its parts.

12.2 The DP recipe

1. **State**: what minimal information describes a subproblem? (`dp[i]`, `dp[i][j]`, `dp[i][w]`...) Write its meaning in one sentence.
2. **Transition**: how does a state's answer come from smaller states? (the recurrence)
3. **Base cases**: the smallest states whose answers are known directly.
4. **Order**: compute states so that dependencies are ready (loop direction).
5. **Answer and optimization**: which state holds the final answer? Can old rows/values be discarded to save space?

Top-down (memoization): write the natural recursion and cache results (`@lru_cache`). Easy to write; recursion depth can be a limit. **Bottom-up (tabulation)**: fill a table iteratively from the base cases. Usually faster, no recursion, and easy to compress to O(1) or O(n) space.

One problem, three styles: climbing stairs

You climb 1 or 2 steps at a time. Ways to reach step i = ways to reach i-1 plus ways to reach i-2 (the last move was a 1-step or a 2-step). Because only the last two values matter, the table collapses to two variables.

```python
from functools import lru_cache


def climb_memo(n):                          # top-down: O(n) time, O(n) space
    @lru_cache(maxsize=None)
    def f(i):
        if i <= 1:
            return 1
        return f(i - 1) + f(i - 2)
    return f(n)


def climb_tab(n):                           # bottom-up: O(n) time, O(n) space
    dp = [1] * (n + 1)
    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]
    return dp[n]


def climb_opt(n):                           # space-optimized: O(n) time, O(1) space
    a, b = 1, 1
    for _ in range(n - 1):
        a, b = b, a + b
    return b
```

TIME O(n) **SPACE** O(n) -> O(1)

12.3 1D dynamic programming

House robber

At each house either rob it (then you must have skipped the previous house) or skip it. Track two numbers: best total if the latest house is robbed (`take`) and best total if it is skipped (`skip`).

```python
def rob(nums):
    take, skip = 0, 0
    for x in nums:
        take, skip = skip + x, max(take, skip)
    return max(take, skip)
```

TIME O(n) **SPACE** O(1)

Coin change: fewest coins, and number of ways

Fewest coins: `dp[a]` = min over coins c of `dp[a - c] + 1`. **Number of ways**: put coins in the *outer* loop so each combination is counted once (swapping loops would count different orders as different).

```python
def coin_change(coins, amount):
    INF = amount + 1
    dp = [0] + [INF] * amount
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a:
                dp[a] = min(dp[a], dp[a - c] + 1)
    return dp[amount] if dp[amount] != INF else -1


def coin_ways(coins, amount):
    dp = [1] + [0] * amount
    for c in coins:                          # coins outer => combinations
        for a in range(c, amount + 1):
            dp[a] += dp[a - c]
    return dp[amount]
```

TIME O(amount * coins) **SPACE** O(amount)

Longest increasing subsequence (O(n^2) and O(n log n))

O(n^2) DP: `dp[i]` = longest increasing subsequence ending at i = 1 + best dp[j] for j < i with a smaller value. **O(n log n)** patience trick: `tails[k]` is the smallest possible tail of an increasing subsequence of length k+1. Each new number either extends the longest one or replaces the first tail that is >= it, found with binary search.

```python
import bisect


def lis_n2(nums):
    if not nums:
        return 0
    dp = [1] * len(nums)
    for i in range(len(nums)):
        for j in range(i):
            if nums[j] < nums[i]:
                dp[i] = max(dp[i], dp[j] + 1)
    return max(dp)


def lis_nlogn(nums):
    tails = []                               # tails[k] = smallest tail of length k+1
    for x in nums:
        i = bisect.bisect_left(tails, x)
        if i == len(tails):
            tails.append(x)
```

```
        else:
            tails[i] = x
    return len(tails)
```

TIME O(n^2) / O(n log n) **SPACE** O(n)

Word break and decode ways

Word break: `dp[i]` is True if some earlier `dp[j]` is True and `s[j:i]` is a word. **Decode ways** ('12' can be AB or L): the count at position i adds ways from a valid single digit and from a valid two-digit number 10 to 26. Two variables suffice.

```
def word_break(s, words):
    words = set(words)
    dp = [False] * (len(s) + 1)
    dp[0] = True
    for i in range(1, len(s) + 1):
        for j in range(i):
            if dp[j] and s[j:i] in words:
                dp[i] = True
                break
    return dp[-1]


def num_decodings(s):
    if not s or s[0] == "0":
        return 0
    prev2, prev1 = 1, 1                      # ways for s[:i-2] and s[:i-1]
    for i in range(1, len(s)):
        cur = 0
        if s[i] != "0":
            cur += prev1
        if 10 <= int(s[i - 1:i + 1]) <= 26:
            cur += prev2
        prev2, prev1 = prev1, cur
    return prev1
```

TIME O(n^2) / O(n) **SPACE** O(n) / O(1)

Stock profit and maximum product subarray

Best time to buy and sell once: track the lowest price so far; profit today = price - lowest. **Maximum product subarray**: a negative number can turn the smallest product into the largest, so keep both the running max and min.

```
def max_profit(prices):
    lowest, best = float("inf"), 0
    for p in prices:
        lowest = min(lowest, p)
        best = max(best, p - lowest)
    return best


def max_product(nums):
    hi = lo = best = nums[0]
    for x in nums[1:]:
        cand = (x, hi * x, lo * x)
        hi, lo = max(cand), min(cand)
        best = max(best, hi)
    return best
```

TIME O(n) **SPACE** O(1)

12.4 2D dynamic programming on grids

Unique paths and minimum path sum

A cell can only be entered from above or from the left, so `dp[r][c]` depends on those two neighbours. Each row only needs the previous row, so one 1D array is enough (the value already in `dp[c]` is 'from above', `dp[c-1]` is 'from the left').

```python
def unique_paths(m, n):
    dp = [1] * n
    for _ in range(1, m):
        for j in range(1, n):
            dp[j] += dp[j - 1]
    return dp[-1]


def min_path_sum(grid):
    R, C = len(grid), len(grid[0])
    dp = [float("inf")] * C
    dp[0] = 0
    for r in range(R):
        dp[0] += grid[r][0]
        for c in range(1, C):
            dp[c] = min(dp[c], dp[c - 1]) + grid[r][c]
    return dp[-1]
```

TIME O(rows * cols) **SPACE** O(cols)

12.5 Knapsack family

You have items with weights and values and a bag of limited capacity. **0/1 knapsack**: each item can be used once. **Unbounded knapsack**: unlimited copies. The only difference in the 1D version is the **direction of the capacity loop**: *backwards* for 0/1 (so an item cannot be reused in the same pass), *forwards* for unbounded (reuse allowed).

0/1 knapsack, unbounded knapsack and partition equal subset

Partition equal subset sum is knapsack in disguise: can some subset reach exactly half of the total? `dp[s]` says 'sum s is reachable'.

```python
def knapsack_01(weights, values, cap):
    dp = [0] * (cap + 1)
    for w, v in zip(weights, values):
        for c in range(cap, w - 1, -1):      # backwards: each item once
            dp[c] = max(dp[c], dp[c - w] + v)
    return dp[cap]


def knapsack_unbounded(weights, values, cap):
    dp = [0] * (cap + 1)
    for w, v in zip(weights, values):
        for c in range(w, cap + 1):          # forwards: items reusable
            dp[c] = max(dp[c], dp[c - w] + v)
    return dp[cap]


def can_partition(nums):
    total = sum(nums)
    if total % 2:
        return False
    target = total // 2
    dp = [True] + [False] * target
    for x in nums:
        for s in range(target, x - 1, -1):
            dp[s] = dp[s] or dp[s - x]
    return dp[target]
```

TIME O(n * capacity) **SPACE** O(capacity)

12.6 String dynamic programming

Longest common subsequence (LCS)

`dp[i][j]` = LCS of the first i letters of a and the first j letters of b. If the last letters match, extend the diagonal; otherwise take the better of dropping a letter from either string. Also the base of diff tools and DNA comparison.

```
def lcs(a, b):
    m, n = len(a), len(b)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
    return dp[m][n]
```

TIME O(m * n) **SPACE** O(m * n)

Edit distance (Levenshtein)

Minimum insert, delete or replace operations to turn a into b. If the letters match, no cost: copy the diagonal. Otherwise 1 + the best of replace (diagonal), delete (up) or insert (left). Rolling one row keeps space at O(n).

```
def edit_distance(a, b):
    m, n = len(a), len(b)
    dp = list(range(n + 1))                 # row for the empty prefix of a
    for i in range(1, m + 1):
        prev, dp[0] = dp[0], i              # prev holds dp[i-1][j-1]
        for j in range(1, n + 1):
            cur = dp[j]
            if a[i - 1] == b[j - 1]:
                dp[j] = prev
            else:
                dp[j] = 1 + min(prev, dp[j], dp[j - 1])
            prev = cur
    return dp[n]


def longest_palindromic_subsequence(s):
    return lcs(s, s[::-1])                  # reuses lcs() from above
```

TIME O(m * n) **SPACE** O(n)

12.7 Bitmask DP

Travelling salesman (visit every city once, return home)

State = (set of visited cities as a bitmask, current city). `dp[mask][last]` is the cheapest cost to have visited `mask` ending at `last`. Exponential, but O(n^2 * 2^n) beats the O(n!) brute force and works up to about n = 20.

```
def tsp(dist):
    n = len(dist)
    INF = float("inf")
    dp = [[INF] * n for _ in range(1 << n)]
    dp[1][0] = 0                            # start at city 0, only city 0 visited
    for mask in range(1 << n):
        for last in range(n):
            if dp[mask][last] == INF:
                continue
            for nxt in range(n):
                if mask & (1 << nxt):
```

```
                continue                # already visited
            nm = mask | (1 << nxt)
            cost = dp[mask][last] + dist[last][nxt]
            if cost < dp[nm][nxt]:
                dp[nm][nxt] = cost
    full = (1 << n) - 1
    return min(dp[full][i] + dist[i][0] for i in range(n))
```

TIME O(n^2 * 2^n) **SPACE** O(n * 2^n)

12.8 DP pattern table

Pattern	State	Examples
Linear / sequence	dp[i] = best for the first i items	Climbing stairs, house robber, LIS, decode ways, word break
Grid	dp[r][c] = best to reach cell (r, c)	Unique paths, min path sum, maximal square
Knapsack	dp[i][w] or dp[w] = best with capacity w	0/1 knapsack, coin change, partition subset sum
Two strings	dp[i][j] = answer for prefixes a[:i], b[:j]	LCS, edit distance, distinct subsequences
Interval	dp[l][r] = best for the segment l..r	Matrix chain, burst balloons, palindrome partitioning
Bitmask	dp[mask][last]	Travelling salesman, assignment problems
Tree / DAG	dp[node] combined from children	Max path sum, house robber III, longest path in a DAG

Debugging a DP. (1) Write the meaning of `dp[...]` in words. (2) Fill the table by hand for a tiny input. (3) Print the table and compare. Most DP bugs are wrong base cases or the wrong loop direction.

CHAPTER 13

Greedy Algorithms

Make the best-looking choice right now and never look back.

A **greedy algorithm** builds a solution step by step, always taking the locally best option. It is fast (usually O(n log n) because of a sort or heap) and simple, but it is only correct when the problem has the **greedy-choice property**: a local best choice can always be extended to a global best. Coin change with coins {1, 3, 4} and amount 6 shows the danger: greedy picks 4+1+1 (three coins) but 3+3 (two coins) is better, so that problem needs DP.

13.1 How to convince yourself greedy works

- **Exchange argument**: take any optimal solution; show you can swap its first choice for the greedy choice without making it worse.
- **Stays ahead**: show that after every step greedy is at least as good as any other strategy.
- **Test it**: compare against brute force on small random inputs. If it ever fails, it is not greedy-solvable.

13.2 Interval problems

Merge intervals

Sort by start time. Then each interval either overlaps the last merged one (extend its end) or starts a new group.

```python
def merge_intervals(intervals):
    intervals.sort(key=lambda x: x[0])
    merged = []
    for s, e in intervals:
        if merged and s <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], e)
        else:
            merged.append([s, e])
    return merged
```

TIME O(n log n) **SPACE** O(n)

Activity selection: maximum non-overlapping intervals

Sort by **end time** and always take the interval that finishes earliest and does not overlap the last chosen one. Finishing early leaves the most room for the rest (sorting by start or by length is wrong). The answer to 'minimum intervals to remove' is `n - max_non_overlapping`.

```python
def max_non_overlapping(intervals):
    intervals.sort(key=lambda x: x[1])       # sort by END time
    count, end = 0, float("-inf")
    for s, e in intervals:
        if s >= end:
            count += 1
            end = e
    return count
```

TIME O(n log n) **SPACE** O(1)

Meeting rooms II: minimum rooms needed

Sort meetings by start. A min-heap holds the end time of every room in use. If the earliest-ending room is free by the time a meeting starts, reuse it; otherwise open a new room. The heap size at the end is the answer.

```python
import heapq


def min_meeting_rooms(intervals):
    intervals.sort()
    heap = []                                   # end times of rooms in use
    for s, e in intervals:
        if heap and heap[0] <= s:
            heapq.heapreplace(heap, e)          # reuse a room
        else:
            heapq.heappush(heap, e)             # need a new room
    return len(heap)
```

TIME O(n log n) **SPACE** O(n)

13.3 Reachability and circuits

Jump game: can you reach the end, and in how few jumps?

Track the farthest index reachable so far. If you ever stand beyond it, you are stuck. For the minimum jumps, treat reachable ranges as BFS levels: each time you pass the end of the current range, you must take one more jump.

```python
def can_jump(nums):
    reach = 0
    for i, x in enumerate(nums):
        if i > reach:
            return False
        reach = max(reach, i + x)
    return True


def min_jumps(nums):
    jumps = cur_end = farthest = 0
    for i in range(len(nums) - 1):
        farthest = max(farthest, i + nums[i])
        if i == cur_end:                        # finished the current range
            jumps += 1
            cur_end = farthest
    return jumps
```

TIME O(n) **SPACE** O(1)

Gas station (circular tour)

If total gas is less than total cost, no start works. Otherwise a solution exists: whenever the running tank goes negative at station i, no station before i can be the start, so restart from i + 1.

```python
def can_complete_circuit(gas, cost):
    if sum(gas) < sum(cost):
        return -1
    start = tank = 0
    for i in range(len(gas)):
        tank += gas[i] - cost[i]
        if tank < 0:
            start, tank = i + 1, 0
    return start
```

TIME O(n) **SPACE** O(1)

13.4 Value-density and coding

Fractional knapsack

When items can be split, take items in decreasing *value per weight* order and fill the last one partially. (Contrast with 0/1 knapsack, where greedy fails and DP is needed.)

```python
def fractional_knapsack(items, cap):          # items: (value, weight)
    items = sorted(items, key=lambda it: it[0] / it[1], reverse=True)
    total = 0.0
    for v, w in items:
        take = min(w, cap)
        total += v * take / w
        cap -= take
        if cap == 0:
            break
    return total
```

TIME O(n log n) **SPACE** O(n)

Huffman coding (optimal prefix codes)

Frequent symbols get short bit strings, rare ones long strings. Repeatedly merge the two least frequent nodes into a parent (heap), until one tree remains; the path from the root (0 = left, 1 = right) is each symbol's code. No code is a prefix of another, so decoding is unambiguous. A counter breaks ties so tuples never compare nodes.

```python
import heapq
from itertools import count


def huffman_codes(freq):                      # freq: {symbol: count}
    tie = count()
    heap = [(f, next(tie), ch) for ch, f in freq.items()]
    heapq.heapify(heap)
    if len(heap) == 1:
        return {heap[0][2]: "0"}
    while len(heap) > 1:
        f1, _, a = heapq.heappop(heap)
        f2, _, b = heapq.heappop(heap)
        heapq.heappush(heap, (f1 + f2, next(tie), (a, b)))
    codes = {}

    def walk(node, code):
        if isinstance(node, str):
            codes[node] = code
        else:
            walk(node[0], code + "0")
            walk(node[1], code + "1")

    walk(heap[0][2], "")
    return codes
```

TIME O(n log n) **SPACE** O(n)

Greedy cheat sheet. Intervals: sort by start to merge, by end to select. Resource scheduling: sort plus min-heap. Reachability: track the farthest point. Optimal merging/encoding: min-heap (Huffman). If a counter-example appears, switch to DP.

CHAPTER 14

Advanced Structures: Trie, Fenwick, Segment Tree, KMP

Specialized tools for prefix search, range queries and fast string matching.

14.1 Trie (prefix tree)

A **trie** stores strings character by character along tree paths, so all words sharing a prefix share the same starting path. Insert, search and prefix lookup cost O(L) for a word of length L, independent of how many words are stored. Perfect for autocomplete, spell check and word-search puzzles.

```
words: app, apple, apt, bat

root --a--> (a) --p--> (p) --p--> (p*) --l--> (l) --e--> (e*)
 |                      \
 |                       +--t--> (t*)
 +--b--> (b) --a--> (a) --t--> (t*)          * = end of a word
```

Trie with insert, search, prefix and autocomplete

Each node holds a dictionary of children and an `end` flag. `search` requires the flag; `starts_with` only requires that the path exists.

```python
class TrieNode:
    def __init__(self):
        self.children = {}
        self.end = False


class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):                  # O(L)
        node = self.root
        for ch in word:
            node = node.children.setdefault(ch, TrieNode())
        node.end = True

    def _walk(self, s):
        node = self.root
        for ch in s:
            node = node.children.get(ch)
            if node is None:
                return None
        return node

    def search(self, word):                  # O(L)
        node = self._walk(word)
        return node is not None and node.end

    def starts_with(self, prefix):           # O(L)
        return self._walk(prefix) is not None

    def words_with_prefix(self, prefix):     # autocomplete
        node, out = self._walk(prefix), []

        def dfs(n, path):
            if n.end:
                out.append(prefix + path)
```

```
            for ch, nxt in sorted(n.children.items()):
                dfs(nxt, path + ch)

        if node:
            dfs(node, "")
        return out
```

TIME O(L) per operation **SPACE** O(total characters)

14.2 Range queries with updates

Prefix sums answer range sums in O(1) but break if a value changes (rebuilding costs O(n)). When the array is **updated** between queries, use a Fenwick tree or segment tree: both give O(log n) updates and O(log n) range queries.

Fenwick tree (Binary Indexed Tree)

Short and fast. Each slot stores the sum of a block whose size is the lowest set bit of its index. `i & -i` isolates that lowest bit and drives both the update climb and the query descent. Supports sums (and any invertible operation).

```
class Fenwick:
    def __init__(self, n):
        self.n = n
        self.t = [0] * (n + 1)

    def add(self, i, delta):                # i is 0-based; O(log n)
        i += 1
        while i <= self.n:
            self.t[i] += delta
            i += i & -i

    def prefix(self, i):                    # sum of a[0..i]; O(log n)
        i += 1
        s = 0
        while i > 0:
            s += self.t[i]
            i -= i & -i
        return s

    def range_sum(self, l, r):              # sum of a[l..r]
        return self.prefix(r) - (self.prefix(l - 1) if l > 0 else 0)
```

TIME O(log n) per operation **SPACE** O(n)

Segment tree (iterative, range sum with point update)

A complete binary tree stored in an array of size 2n. Leaves (positions n to 2n-1) hold the data; every inner node stores the combination (here the sum) of its two children. Swap `+` for `min`, `max` or `gcd` to get other range queries. Unlike a Fenwick tree it handles non-invertible operations like min and max.

```
class SegmentTree:
    def __init__(self, nums):
        self.n = len(nums)
        self.t = [0] * self.n + list(nums)
        for i in range(self.n - 1, 0, -1):
            self.t[i] = self.t[2 * i] + self.t[2 * i + 1]

    def update(self, i, val):               # set nums[i] = val; O(log n)
        i += self.n
        self.t[i] = val
        while i > 1:
            i //= 2
            self.t[i] = self.t[2 * i] + self.t[2 * i + 1]

    def query(self, l, r):                  # sum of nums[l..r]; O(log n)
```

```
        l += self.n
        r += self.n + 1
        s = 0
        while l < r:
            if l & 1:
                s += self.t[l]
                l += 1
            if r & 1:
                r -= 1
                s += self.t[r]
            l //= 2
            r //= 2
        return s
```

TIME build O(n), update/query O(log n) **SPACE** O(n)

14.3 Fast string matching

KMP: find a pattern inside a text in linear time

Naive matching restarts from scratch after a mismatch (O(n * m)). KMP precomputes, for every prefix of the pattern, the length of its longest proper prefix that is also a suffix (the `lps` table). On a mismatch it jumps the pattern pointer back using that table and **never moves backwards in the text**.

```
def kmp_search(text, pat):                  # returns all start indices
    if not pat:
        return []
    lps = [0] * len(pat)                    # longest proper prefix that is also a suffix
    k = 0
    for i in range(1, len(pat)):
        while k and pat[i] != pat[k]:
            k = lps[k - 1]
        if pat[i] == pat[k]:
            k += 1
        lps[i] = k
    res, k = [], 0
    for i, ch in enumerate(text):
        while k and ch != pat[k]:
            k = lps[k - 1]
        if ch == pat[k]:
            k += 1
        if k == len(pat):
            res.append(i - k + 1)
            k = lps[k - 1]
    return res
```

TIME O(n + m) **SPACE** O(m)

Need	Structure	Update	Query
Prefix search, autocomplete	Trie	O(L)	O(L)
Static range sums	Prefix sums	n/a (rebuild O(n))	O(1)
Dynamic range sums	Fenwick tree	O(log n)	O(log n)
Dynamic range min/max/gcd	Segment tree	O(log n)	O(log n)
Substring search	KMP (or Python's `in`, which is optimized)	n/a	O(n + m)

CHAPTER 15

Bit Manipulation

Work directly on the binary digits of numbers: tiny, fast and surprisingly powerful.

Operator	Meaning	Example
a & b	AND: 1 only if both bits are 1	6 & 3 = 2 (110 & 011 = 010)
a | b	OR: 1 if either bit is 1	6 | 3 = 7 (110 | 011 = 111)
a ^ b	XOR: 1 if the bits differ	6 ^ 3 = 5 (110 ^ 011 = 101)
~a	NOT: flip all bits	~5 = -6 in Python
a << k	Shift left = multiply by 2^k	3 << 2 = 12
a >> k	Shift right = floor-divide by 2^k	13 >> 2 = 3

15.1 Three XOR facts that solve many puzzles

- `x ^ x = 0` (a number cancels itself), `x ^ 0 = x`, and XOR is commutative and associative.
- So XOR-ing a list in which every number appears twice except one leaves exactly the odd one out.
- `n & (n - 1)` clears the lowest set bit. It is zero only for powers of two (and 0).

The essential bit tricks

Study each trick on paper with binary numbers. `1 << i` builds a mask with only bit i set; AND tests, OR sets, AND-NOT clears, XOR toggles.

```python
def is_power_of_two(n):
    return n > 0 and (n & (n - 1)) == 0


def count_set_bits(n):                      # Kernighan: loops once per set bit
    c = 0
    while n:
        n &= n - 1                          # clear the lowest set bit
        c += 1
    return c


def single_number(nums):                    # every other value appears twice
    x = 0
    for v in nums:
        x ^= v
    return x


def missing_number(nums):                   # values 0..n with one missing
    x = len(nums)
    for i, v in enumerate(nums):
        x ^= i ^ v
    return x


def get_bit(n, i):
    return (n >> i) & 1


def set_bit(n, i):
    return n | (1 << i)
```

```python
def clear_bit(n, i):
    return n & ~(1 << i)


def toggle_bit(n, i):
    return n ^ (1 << i)


def swap_xor(a, b):
    a ^= b
    b ^= a
    a ^= b
    return a, b
```

TIME O(1) / O(bits) **SPACE** O(1)

Bitmask subsets and reversing bits

A set of n items can be stored in an n-bit integer: bit i set means 'item i is in the set'. Counting from 0 to 2^n - 1 then enumerates all subsets, which is how bitmask DP works.

```python
def all_subsets(nums):                      # O(n * 2^n)
    n = len(nums)
    return [[nums[i] for i in range(n) if (mask >> i) & 1]
            for mask in range(1 << n)]


def reverse_bits32(n):
    res = 0
    for _ in range(32):
        res = (res << 1) | (n & 1)
        n >>= 1
    return res
```

TIME O(n * 2^n) / O(32) **SPACE** O(2^n) / O(1)

Python detail. Python integers have unlimited size and negative numbers behave as if they had infinite two's-complement digits. For 32-bit puzzles, mask with `& 0xFFFFFFFF` to simulate overflow.

CHAPTER 16

Cheat Sheets, Templates and Study Plan

Your quick-reference for recognizing patterns, comparing complexities and staying on track.

16.1 Clue to technique: how to recognize the pattern

If the problem says...	Think of...	Chapter
Sorted array, find a pair or target	Two pointers or binary search	Arrays, Searching
Longest/shortest subarray or substring with a property	Sliding window	Arrays
Count/find duplicates, pairs, frequency, anagram	Hash map or set	Hashing
Range sum queries on a static array	Prefix sums	Arrays
Next greater/smaller, histogram, matching brackets	Monotonic stack / stack	Stacks and Queues
Linked list middle, cycle, k-th from end	Slow and fast pointers	Linked Lists
Generate all combinations, permutations, subsets, boards	Backtracking	Recursion
Minimize the maximum / maximize the minimum	Binary search on the answer	Searching
K largest, K smallest, K most frequent, merge K	Heap	Heaps
Shortest path, fewest steps, levels	BFS (weighted: Dijkstra)	Graphs
Connected components, islands, cycles	DFS/BFS or Union-Find	Graphs
Dependencies, ordering, prerequisites	Topological sort	Graphs
Number of ways, min/max cost with choices, overlapping subproblems	Dynamic programming	DP
Intervals, scheduling, 'take the best now'	Greedy (sort first)	Greedy
Prefix lookups, dictionary of words	Trie	Advanced
Updates and range queries together	Fenwick / segment tree	Advanced
Find the single odd one, subsets of small n, flags	Bit manipulation	Bit Manipulation
Tree: depth, path, ancestor	DFS recursion; return info to parent	Trees
n up to 20, all subsets matter	Bitmask or backtracking	DP, Bit Manipulation

16.2 Data structure operations at a glance

Structure	Access	Search	Insert	Delete	Space
Array / list	O(1)	O(n)	O(n) (end: O(1))	O(n)	O(n)
Linked list	O(n)	O(n)	O(1) at head	O(1) if node known	O(n)
Stack / queue / deque	O(n)	O(n)	O(1)	O(1)	O(n)
Hash table	n/a	O(1) avg	O(1) avg	O(1) avg	O(n)
Binary search tree (balanced)	O(log n)	O(log n)	O(log n)	O(log n)	O(n)
Binary search tree (skewed)	O(n)	O(n)	O(n)	O(n)	O(n)
Heap (binary)	O(1) min	O(n)	O(log n)	O(log n) min	O(n)
Trie	n/a	O(L)	O(L)	O(L)	O(total chars)
Union-Find	n/a	~O(1) find	~O(1) union	n/a	O(n)
Fenwick / segment tree	O(log n)	n/a	O(log n) update	n/a	O(n)

16.3 Algorithm complexities at a glance

Algorithm	Time	Space
Binary search	O(log n)	O(1)
Merge sort / heap sort	O(n log n)	O(n) / O(1)
Quick sort	O(n log n) avg, O(n^2) worst	O(log n)
BFS / DFS	O(V + E)	O(V)
Dijkstra (heap)	O((V + E) log V)	O(V)
Bellman-Ford	O(V * E)	O(V)
Floyd-Warshall	O(V^3)	O(V^2)
Kruskal / Prim	O(E log E) / O(E log V)	O(V + E)
Topological sort	O(V + E)	O(V)
0/1 knapsack	O(n * W)	O(W)
LCS / edit distance	O(m * n)	O(min(m, n)) rolling, O(m * n) full
LIS	O(n log n)	O(n)
KMP	O(n + m)	O(m)
Subsets / permutations	O(2^n) / O(n!)	O(n)

16.4 Template vault

Skeletons to adapt. The *shape* of the solution stays the same; only the condition and the bookkeeping change.

```python
# 1) Binary search on the answer: smallest x for which feasible(x) is True
def smallest_feasible(lo, hi):
    while lo < hi:
        mid = (lo + hi) // 2
        if feasible(mid):
            hi = mid
        else:
            lo = mid + 1
    return lo


# 2) Variable-size sliding window
def longest_window(arr):
    left = 0
```

```
    for right, x in enumerate(arr):
        add(x)                              # put arr[right] into the window state
        while window_is_invalid():
            remove(arr[left])
            left += 1
        best = max(best, right - left + 1)


# 3) BFS by levels (shortest path in steps)
def bfs_levels(start):
    q, seen, steps = deque([start]), {start}, 0
    while q:
        for _ in range(len(q)):
            u = q.popleft()
            if is_goal(u):
                return steps
            for v in neighbours(u):
                if v not in seen:
                    seen.add(v)
                    q.append(v)
        steps += 1
    return -1


# 4) Backtracking
def backtrack(path):
    if is_solution(path):
        result.append(path[:])
        return
    for choice in choices(path):
        if not valid(choice):
            continue
        path.append(choice)
        backtrack(path)
        path.pop()


# 5) Top-down DP
@lru_cache(maxsize=None)
def solve(state):
    if is_base(state):
        return base_value(state)
    return combine(solve(next_state) for next_state in transitions(state))
```

16.5 Practice problems by topic

Solve these in order within each topic. Search the exact title on LeetCode, GeeksforGeeks or any judge. Aim to solve, then re-solve from memory a week later.

Topic	Problems (easy to hard)
Arrays and strings	Two Sum II, Move Zeroes, Best Time to Buy and Sell Stock, Maximum Subarray, Product of Array Except Self, 3Sum, Longest Substring Without Repeating Characters, Minimum Window Substring
Hashing	Contains Duplicate, Valid Anagram, Group Anagrams, Subarray Sum Equals K, Longest Consecutive Sequence, LRU Cache
Linked lists	Reverse Linked List, Merge Two Sorted Lists, Linked List Cycle, Remove Nth Node From End, Add Two Numbers, Reorder List, Merge k Sorted Lists
Stacks and queues	Valid Parentheses, Min Stack, Daily Temperatures, Evaluate Reverse Polish Notation, Largest Rectangle in Histogram, Sliding Window Maximum
Recursion / backtracking	Subsets, Permutations, Combination Sum, Generate Parentheses, Word Search, N-Queens, Sudoku Solver
Searching and sorting	Binary Search, Search in Rotated Sorted Array, Find First and Last Position, Koko Eating Bananas, Median of Two Sorted Arrays, Kth Largest Element

Topic	Problems (easy to hard)
Trees and BST	Maximum Depth, Invert Binary Tree, Level Order Traversal, Validate BST, Lowest Common Ancestor, Binary Tree Maximum Path Sum, Serialize and Deserialize Binary Tree
Heaps	Kth Largest Element in a Stream, Top K Frequent Elements, K Closest Points to Origin, Task Scheduler, Find Median from Data Stream
Graphs	Number of Islands, Clone Graph, Course Schedule, Rotting Oranges, Network Delay Time, Cheapest Flights Within K Stops, Word Ladder, Redundant Connection
Dynamic programming	Climbing Stairs, House Robber, Coin Change, Longest Increasing Subsequence, Word Break, Unique Paths, Longest Common Subsequence, Edit Distance, Partition Equal Subset Sum
Greedy	Assign Cookies, Jump Game, Merge Intervals, Non-overlapping Intervals, Gas Station, Meeting Rooms II
Advanced	Implement Trie, Range Sum Query Mutable, Find All Anagrams, Find the Index of the First Occurrence in a String
Bit manipulation	Single Number, Number of 1 Bits, Power of Two, Missing Number, Reverse Bits, Subsets via bitmask

16.6 A 12-week study plan

Week	Focus	Goal
1	Complexity analysis, Python toolkit, arrays	Explain Big-O; solve 10 easy array problems
2	Strings, two pointers, sliding window, prefix sums	Recognize the three patterns on sight
3	Hashing, linked lists	Implement a hash map and every linked-list technique from memory
4	Stacks, queues, monotonic stack	Solve the 'next greater' family without hints
5	Recursion and backtracking	Write the backtracking template from memory
6	Binary search (all variants), sorting	Code merge sort and quick sort; solve binary-search-on-answer
7	Trees and BST	All traversals recursive and iterative; LCA; validate BST
8	Heaps and priority queues	Top-K patterns, merge K lists, running median
9	Graphs I: BFS, DFS, components, topological sort	Solve grid and dependency problems
10	Graphs II: Dijkstra, Union-Find, MST	Pick the right shortest-path algorithm by constraints
11	Dynamic programming	Do 25 DP problems, write the state in words first every time
12	Greedy, trie, bit manipulation, mixed review	Timed mock interviews: 2 problems in 60 minutes

Study rhythm. One new concept at a time, 1 to 2 hours a day. After each topic do 5 problems; one week later redo two of them without looking. Spaced repetition beats cramming. Keep a notebook of the mistakes you made and review it before every mock interview.

16.7 Common mistakes checklist

- **Off-by-one**: decide whether ranges are inclusive `[lo, hi]` or half-open `[lo, hi)` and stay consistent; use the matching loop condition.
- **Empty and single-element inputs**: test them first.
- **Mutating while iterating**: do not add or remove from a list you are looping over.
- **Forgetting to copy** in backtracking (`path[:]`) or aliasing 2D lists (`[[0]*n]*m`).
- **Hidden O(n)**: `list.pop(0)`, `x in list`, string concatenation in a loop, slicing inside a loop.
- **Missing visited set** in graph traversal (infinite loop) or marking visited too late in BFS (duplicates).
- **Integer overflow habits** from other languages; Python ints do not overflow, but float division (`/`) loses precision: use `//` for integers.
- **Not stating complexity**: always finish with 'Time O(...), space O(...) because ...'.

16.8 Where to go next

- **Practice daily** on LeetCode, Codeforces, AtCoder or HackerRank; aim for understanding, not volume.
- **Read**: 'Introduction to Algorithms' (CLRS) for depth; 'Cracking the Coding Interview' and 'Grokking Algorithms' for interviews and intuition.
- **Advanced topics** when you are ready: AVL and Red-Black trees, B-trees, Segment tree with lazy propagation, Sparse table, Tarjan's bridges and articulation points, Max-flow (Ford-Fulkerson, Dinic), Suffix array and automaton, Manacher, Z-algorithm, Mo's algorithm, Convex hull trick, Network flow.
- **Build** something real (a search engine with a trie, a route planner with Dijkstra, a cache with LRU). Using a data structure for a purpose is what makes it stick.

Remember: every expert once wrote their first `for` loop. Consistency beats talent. Good luck, and keep coding.